

Avantgarde: A Machine Learning Based Hybrid Host-Based Intrusion Detection and Prevention System

by

Fardous Nayeem

23341091

Naser-Al-Noman

21301249

Md. Monowarul Islam

23141071

Jotham Protik Baroi

21201626

Nabila Samroj Oishe

20201216

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering and B.Sc. in Computer Science

Department of Computer Science and Engineering
Brac University
October 2025.

© 2025. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Nayeem

Fardous Nayeem

23341091

Naser

Naser-Al-Noman

21301249

Monowarul

Md. Monowarul Islam

23141071

Baroi

Jotham Protik Baroi

21201626

Oishe

Nabila Samroj Oishe

20201216

Approval

The thesis titled “Avantgarde: A Machine Learning Based Hybrid Host-Based Intrusion Detection & Prevention System” submitted by

1. Fardous Nayeem (23341091)
2. Naser-Al-Noman (21301249)
3. Md. Monowarul Islam (23141071)
4. Jotham Protik Baroi (21201626)
5. Nabila Samroj Oishe (20201216)

Of Summer, 2025 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science in October 10, 2025.

Examining Committee:

Supervisor:
(Member)



Md. Sabbir Ahmed

Lecturer
Department of Computer Science and Engineering
Brac University

Co-Supervisor:
(Member)



Muhammad Iqbal Hossain, PhD

Associate Professor
Department of Computer Science and Engineering
Brac University

Thesis Coordinator:
(Member)

Md. Golam Rabiul Alam, PhD

Professor
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Sadia Hamid Kazi, PhD

Chairperson and Associate Professor
Department of Computer Science and Engineering
Brac University

Abstract

In the age of the Internet, cyber threats are increasing at an exponential rate, targeting individuals and institutions of all types. The need for a proper cybersecurity infrastructure integrated with the latest innovations such as artificial intelligence, zero-trust architecture, and automation has become paramount, especially when malicious actors use AI to their advantage. This paper aims to present a HIDPS framework called “Avantgarde” to counter cyber threats by leveraging AI, automation, zero-trust architecture, inclusion of open source tools to create integrated systems with enhanced threat detection, prevention, and response capabilities. Our research aims to assess the effectiveness of this approach and its potential to meet online security requirements. We will discuss the impacts and constraints of the chosen technologies. Then, the Avantgarde design process is explained and reviewed. Lastly, Avantgarde is evaluated on the BCCC-CIC-IDS2017 dataset and live network data. Then analyzed on performance indicators, which shows that Avantgarde’s framework improves the overall accuracy of threat detection, control access, reduction of false positives, and enhanced resilience of a network, highlighting its potential as an adaptive framework capable of addressing modern security challenges in an ever-evolving digital landscape.

Keywords: Intrusion Detection System, Intrusion Prevention System, Host-based Intrusion Detection and Prevention System, Machine Learning, Zero-Trust Architecture, Open-source IDS, Automated Response

Acknowledgement

We are grateful to our supervisor Md. Sabbir Ahmed, Lecturer, BRAC University and to our co-supervisor, Dr. Muhammad Iqbal Hossain, Associate Professor, BRAC University, for their guidance, support and assistance throughout the endeavor. We would like to extend our gratitude to Md. Sabbir Ahmed and Dr. Muhammad Iqbal Hossain for their constant guidance and support throughout our entire journey. We would also like to offer our gratitude to Dr. Md. Sakir Hossain for his immense support throughout our entire research.

Table of Contents

Declaration	i
Approval	ii
Abstract	iv
Acknowledgment	v
Table of Contents	vi
List of Figures	ix
List of Tables	x
Nomenclature	x
1 Introduction	1
1.1 Background	1
1.2 Rational of the Study or Motivation	2
1.3 Problem Statement	2
1.4 Objectives	3
1.5 Methodology in Brief	3
1.6 Summary of Contributions	4
1.7 Scopes and Challenges	5
2 Literature Review	6
2.1 Preliminaries	6
2.2 Review of Existing Research	7
2.2.1 Unified Threat Management	8
2.2.2 ML and DL in Intrusion Detection	9
2.2.3 Host-Based Intrusion Detection Systems (HIDS)	11
2.2.4 Surveys, Benchmarks, and Dataset Realism	12
2.2.5 SIEM and XDR for Scalable Threat Detection	14
2.2.6 Fintech Threat Landscape	15
2.2.7 Real-Time DDoS Detection and Feature Engineering	17
2.2.8 Vulnerability Scanning and Low-Cost Security	17
2.2.9 Automated Response and Reinforcement Learning	18
2.2.10 Generative AI in Cybersecurity	18
2.2.11 Open-Source and Rapid Security Prototyping	19
2.3 Summary of Reviewed Researches	19

3	Requirements, Impacts and Constraints	21
3.1	Final Specifications and Requirements	21
3.2	Societal Impact	21
3.3	Environmental Impact	22
3.4	Ethical Issues	22
3.5	Project Management Plan	22
3.6	Risk Management	23
3.7	Economic Analysis	23
4	Proposed Methodology	24
4.1	Methodology Overview	24
4.2	Preliminary Design Process	26
4.3	Component Functionality	27
4.4	Implementation of Framework	28
4.4.1	Network Data Processing Overview	29
4.4.2	Segmented Access and Management Overview	30
4.4.3	Independent Signature IDS Overview	31
4.4.4	Machine Learning Analysis Engine Overview	31
4.4.5	Runtime Processes	33
4.4.6	Sequence of Operations	33
4.4.7	Performance and Reliability Considerations	35
4.4.8	Prototype Limitations	35
4.4.9	Outcome	36
4.5	Data Collection	37
4.5.1	Data Reduction & Pre-processing	38
4.5.2	Summary of Preprocessed Data	41
4.6	Implementation of Models	42
5	Result Analysis	44
5.1	Performance Evaluation of ML Models	44
5.2	Analysis of Final Design	57
5.2.1	Description of the Final Design	57
5.2.2	Final Design Adjustments	57
5.2.3	Final Design Evaluation	59
5.2.4	Avantgarde Compared to Other Systems	60
5.3	Statistical Analysis	61
5.3.1	Statistical Analysis for Model Evaluation	61
5.3.2	Statistical Model for Decision-Making	62
5.4	Comparisons and Relationships	63
5.4.1	Qualitative Analysis	64
5.4.2	Quantitative Analysis	66
5.5	Discussions	67
5.5.1	Interpretations	67
5.5.2	Implications	68
5.5.3	Limitations	68

6	Conclusion	70
6.1	Summary of Findings	70
6.2	Concluding Remarks	70
6.3	Recommendations for Future Work	71
	Bibliography	76

List of Figures

1.1	Cybercrime Stastics 2024	1
1.2	Workflow Diagram	4
4.1	Avantgarde architecture	25
4.2	Avantgarde Prototype Admin Dashboard	29
4.3	Packet Capturing with Scapy and Model Predictions	30
4.4	Segmentations status of connections	30
4.5	Suricata Running Independently	31
4.6	MLAE WORKFLOW	32
4.7	Extended Details of Decisions	33
4.8	Model predictions on captured Packet IPs	34
4.9	Enforcement by ML Analysis Engine	35
4.10	Initial Dataset Distribution	37
4.11	Feature Importance Graph	38
4.12	Training set Class Distribution Before SMOTE	40
4.13	Training set Class Distribution After SMOTE	40
4.14	Test set distribution after 80:20 splitting	41
4.15	Voting Diagram	43
4.16	Stacking Diagram	43
5.1	Confusion matrix of Random Forest Classifier	44
5.2	Confusion matrix of Random Forest Classifier (Binary Classification)	45
5.3	Confusion Matrix of XGBoost.	46
5.4	Confusion matrix of XGBoost Classifier (Binary Classification)	47
5.5	Confusion Matrix of CatBoost	47
5.6	Confusion matrix of CatBoost Classifier (Binary Classification)	48
5.7	Confusion Matrix StackingRXC [RFC, XGBC, CBC]	50
5.8	Confusion matrix of StackingRXC (Binary Classification)	51
5.9	Confusion Matrix StackingRX [RFC, XGBC]	51
5.10	Confusion matrix of StackingRX (Binary Classification)	52
5.11	Confusion Matrix VotingRX [RFC, XGBC]	53
5.12	Confusion matrix of VotingRX (Binary Classification)	54
5.13	Confusion Matrix VotingRXC [RFC, XGBC, CBC]	55
5.14	Confusion matrix of VotingRXC (Binary Classification)	56

List of Tables

4.1	Selected Features and Their Information Gain	39
5.1	Performance metrics for RandomForest model	45
5.2	Performance metrics for XGBoost model	46
5.3	Performance metrics for CatBoost model	48
5.4	Macro Performance metrics of all trained Models	49
5.5	Binary Performance metrics of all trained Models	49
5.6	Performance Metrics of StackingRXC [RFC, XGBC, CBC]	50
5.7	Performance Metrics of StackingRX [RFC, XGBC]	52
5.8	Performance Metrics of VotingRX [RFC, XGBC]	53
5.9	Performance Metrics of VotingRXC [RFC, XGBC, CBC]	55
5.10	Performance Metrics of the Chosen Models	56
5.11	Evaluation of Avantgarde Design Against Criteria	59
5.12	Evaluation of Avantgarde Design Against Other Systems	60
5.13	Qualitative Analysis of Reviewed Papers	64
5.14	Comparison of related works on IDS using ML/DL	66

Chapter 1

Introduction

1.1 Background

The dependence on the internet in everyday life is ever-rising. Entertainment, education, shopping, banking and socializing, any aspect of life has started to revolve around the internet. In fact 61% of all social or corporate interactions are now held online [49], We have already entered an era where everything is virtualized, dubbed the 4th industrial revolution. With such a broad reach of the internet, the risks and threats to privacy and data security of any entity or individual are ever-increasing. Until 2024, global economic costs from cyber attacks and incidents have reached \$25 billion US dollars[28]. In Figure 1.1, it shows that the cost of ransomware to victims will rise to \$265 billion by 2031[42].

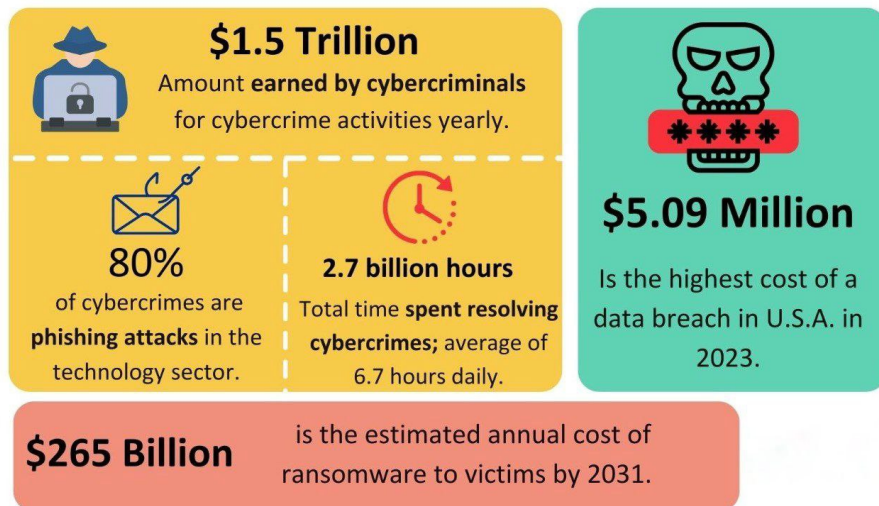


Figure 1.1: Cybercrime Stastics 2024

Data breaches have become a common occurrence as well. In may 2024, Ticketmaster, one of the largest ticket sales and distribution companies fell victim to a cyber attack, leading to the financial and personal data of 560 million customers being breached[50]. Bangladesh is no exception to these trends. The rapid expansion of Bangladesh's general infrastructure and digitization underlines the critical need of cyber-security as it already faces a multitude of cyber attacks daily. A study revealed that 52% Bangladeshi Banks are currently at high risk of cyber attacks with

outdated cyber-security systems [21] despite the infamous 2016 Bangladesh Bank Heist which led to the loss of approximately 81 million USD from the country's foreign reserves [22]. However, in spite of these alarming trends and significant financial losses incurred globally, both business and government entities continue to struggle with outdated cyber-security systems and practices. Traditional reactive practices that rely heavily on known signatures and patterns are proving inadequate against highly sophisticated and evolving threats. Recent advancements in artificial intelligence (AI), blockchain-inspired databases, and innovative industry strategies present the opportunity to protect networks from the increasing dangers imposed by cyber threats. Especially the capabilities of AI such as detecting anomalies by processing large amounts of network traffic adds dynamicity in combating threats in real time. AI can even aid in the recovery of a network with the use of self-healing mechanisms after it has been breached, making its integration crucial in the modern cyberspace.

1.2 Rational of the Study or Motivation

As discussed, everyone is vulnerable to cyber threats. Most fail to deal with such threats as they do not have adequate knowledge to respond to such threats. One of the reasons for any organization to not have cyber-security measures to prevent any attack is due to the immense cost of installing such systems or personnel for an organization. Another reason is that employed systems are outdated and can't detect or deal with modern cyber attacks such as Adversarial AI, Supply Chain Attacks, Advanced Persistent Threats and Zero-day exploits. So there should be focus on developing a system that can detect a variety of intrusions, akin to a Host-based Intrusion Detection and Prevention System (HIDPS). The proposed system should also have the capability to respond to detected intrusions and learn from such events, as well as making sure that overall storage of the logs are done, which can be vital for usage in future. Another beneficial part of such security systems can be their cost-effectiveness of it than most other solutions commonly seen in the traditional landscape of the cybersecurity world, as AI can eliminate the need for supervision of the system by dedicated personnel. Even though the use of AI in cybersecurity has become revolutionary by making the infrastructure stronger, a majority of the tools that are developed can be seen being targeted towards recognising a limited type of threats. There can also be seen a lack of practical implementations through the use of multiple models output for better accuracy in present systems. Apart from using AI, the use of zero-trust architecture and automated response can effectively increase robustness and resilience.

1.3 Problem Statement

The rapid adoption of the internet has exponentially increased the risk of cyber attacks on all sorts of institutions, businesses, individuals. Nowadays, AI can be used to write malicious code and target the public, increasing the number of attackers. These attacks pose significant challenges for traditional rules or signature based intrusion detection systems (IDS) as AI can be used to study the rules and create bypasses of such systems. And tools that do incorporate AI, most of the time such

tools target a single category of attack or their trained behaviors are over reliant on their datasets which causes issues like mislabeling traffic as malicious based on the traffic pattern not resembling traffic pattern in the dataset. So, the current AI incorporated approaches relying heavily on datasets to make decisions which makes them impractical. There is also the matter of inadequate testing of proposed models and systems on live network data, and the observing the discrepancies between detection in dataset testing and live data testing in many papers.

1.4 Objectives

This research aims to advance host-based intrusion detection and prevention by developing a hybrid Host-based Intrusion Detection and Prevention System (HIDPS) framework called *Avantgarde*. The specific objectives of this study are to:

- **Design Framework** as a modular HIDPS that integrates machine learning-based detection, a signature-based intrusion detection system, Zero Trust Architecture principles, and automated response mechanisms.
- **Develop and evaluate** machine learning models for malicious traffic detection using the best suited dataset, optimizing their performance to achieve superior or optimal results compared to existing research works.
- **Implement** a unified decision mechanism that provides the best decision from the independent IDS alert logs and machine learning model predictions.
- **Build and test** a prototype based on the Avantgarde Framework that demonstrates live data capture, anomaly detection, and automated access in terms of access and segmentation.
- **Assess and compare** the performance, reliability, and modular adaptability of the proposed framework and developed models against existing approaches to validate its effectiveness.

1.5 Methodology in Brief

As per our objective, creating a Host-based Intrusion Prevention and Detection System (HIPDS) for network monitoring and security. The BCCC-CIC-IDS2017 [43], CIC-IDS2017 [11], UNSW-NB15 [4] [7] [12] [9] [20] and CAIDA datasets were tested to evaluate different types of attacks in separate environments and how data is presented in respective datasets. Finally, the BCCC-CIC-IDS2017 dataset was chosen for training our chosen models and then building a prototype following the framework architecture, and finally testing it using live network traffic data. The dataset is sanitized using data pre-processing and feature engineering techniques to ensure optimal feature selection for effective threat classification and detection. Supervised machine learning models such as Random Forest, XGBoost, CatBoost, etc for anomaly detection in network traffic. As a result of the discussed methods, the prototype has proven capable in detecting malicious behavior and subsequent response in the chosen network environment while keeping a low false positive rates. The proficiency of the models are represented using suitable benchmarks including

accuracy, macro-precision, macro-recall, macro-f1-score. The desired outcome of our research is to develop a framework that is modular and automated that can successfully combat malicious behaviours by integrating open-source ids, machine-learning and zero trust architecture.

Our workflow details the step by step process of researching, building and testing a prototype outlined by the proposed framework for this study. From initial research of existing systems and research papers, reviewing and testing multiple datasets, categorically testing various ML models and choosing the optimal ones. Researching and developing the prototype of Avantgarde. Then testing the prototype system in a simulated network environment. Finally evaluate the performance and showcase analysis based on it. The following figure presents the workflow:

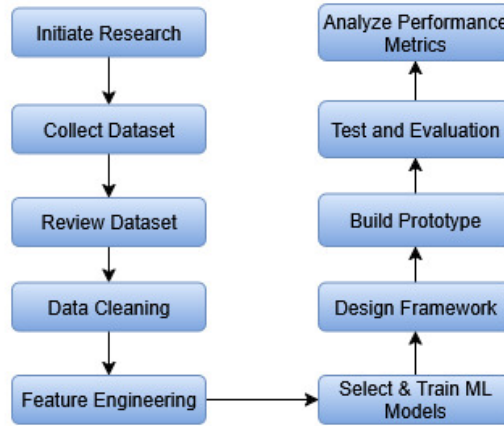


Figure 1.2: Workflow Diagram

1.6 Summary of Contributions

This research contributes to the advancement of host-based intrusion detection and prevention systems by proposing a modular and adaptive framework that addresses security challenges in host devices by achieving the following:

- Designed the Avantgarde Framework, a modular HIDPS architecture integrating machine learning-based intrusion detection, a signature-driven IDS (e.g., Suricata), Zero Trust Architecture principals, and automated response mechanism for adaptive threat detection and prevention.
- Achieved higher recall and f1-scores with developed models compared to previous research using the BCCC-CIC-IDS2017 dataset, demonstrating practical viability for usage in the framework.
- Implemented a statistical fusion decision feature using an Exponential Moving Average (EMA) based confidence score to continuously refine and stabilize intrusion verdicts over time.
- Developed a prototype integrating real-time data capture, ML-based anomaly detection and automated response in terms of access and segmentation.

1.7 Scopes and Challenges

The scope of this paper is to design and develop 'Avantgarde', the proposed Host-based Intrusion Detection and Prevention System integrating Artificial Intelligence, Response Automation and Zero Trust Architecture to safeguard any given network. The framework is designed for adaptability and quick response which makes it scalable in any environment as well as being able to face a multitude of attacks ensuring optimal protection of the network. Despite its great potential, there are several challenges in the development of Avantgarde. The most daunting one being the integration of an independent intrusion detection system into the prototype to build a single cohesive system, which requires thorough testing to ensure its feasibility and as well as optimal efficiency. Another problem that arises from relying on ML models for detecting anomalies presents issues such as false positives, overfitting, which causes the system to identify all attacks as specific attacks despite not being the case. Making decisions based on the outputs of both the independent intrusion detection and the machine learning analysis engine also has caveats such as latency and overhead complexities which may affect the overall stability of the prototype. Despite its effectiveness, the Zero-Trust architecture requires constant authentication and the segmentation of the network, further increasing the framework design complexity. The automated response feature can also malfunction and block benign traffic, disrupting the network. Retraining machine learning analysis engine and the rate of retraining can also cause disruptions as well as increasing the false positive rate due to inadequate retraining or data samples.

Chapter 2

Literature Review

2.1 Preliminaries

These are some concepts, and theories which are given to readers to make key terms easier to understand in the literature review:

- **Zero Trust Architecture (ZTA):** ZTA is a security design approach where strict ID verification, least-privilege access and continuous monitoring are enforced on all users or devices, inside or outside the network as no device or user is trusted by default.
- **Intrusion Detection System (IDS):** IDS is a security tool to monitor network traffic for identifying threats by detecting signs of known threats or by anomalies based on pretrained datasets.
- **Intrusion Prevention System (IPS):** IPS works as an extension to the IDS by taking automated actions based on the threats detected by the IDS.
- **Host-based Intrusion Detection and Prevention System (HIDPS):** HIDPS is a security control installed on individual hosts that continuously monitors network activity. When suspicious behavior or known attack patterns are detected, HIDPS not only generates alerts but can actively intervene, blocking or containing malicious actions. Thus combining both detection and prevention at the host level.
- **Unified Threat Management (UTM):** UTM is a cybersecurity solution that consists of multiple features such as firewall, antivirus, intrusion detection and content filtering all in a single platform.
- **Automated Response:** Automated response is a security mechanism where predefined actions are automatically triggered once a threat or anomaly is detected, minimizing human intervention by enforcing containment, mitigation, or recovery steps in real time, ensuring faster reaction to incidents and reducing the window of exploitation.
- **Ensemble Learning (EL):** EL is a ML technique that combines and uses two or more ML models to improve the accuracy of predictions. In cybersecurity, EL is usually used in IDS to improve accuracy of threat detection.

- **Deep Learning (DL):** DL is a subset of ML that uses a multilayered neural network to perform tasks like data classification, regression and representation learning. In cybersecurity, deep learning is useful for tasks like detecting malware and phishing attacks and analysing network traffic.
- **Reinforcement Learning (RL):** RL is a form of ML where an AI agent is trained to take desirable actions based on a reward or penalty based system as a feedback to the actions. RL is usually used for adaptive response and resource allocation in cybersecurity.
- **Federated Learning (FL):** FL is a collaborative ML approach where multiple devices or nodes train a single model, but does not share any raw data in the process. In cybersecurity, FL is usually used in threat detection and analysis while preserving privacy of data.
- **Security Information and Event Management (SIEM):** SIEM is a security solution that works by collecting and correlating network logs and event data to help detect and prevent cyber threats in real-time.
- **Extended Detection and Response (XDR):** XDR is a unified platform that collects and correlates data from multiple security layers (such as; end-points, network, cloud) to automate detection, investigation, and response to security threats.

2.2 Review of Existing Research

This literature review discusses topics considered the corners of modern cyber security such as Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS), Unified Threat Management (UTM), Host-based Intrusion Detection and Prevention Systems (HIDPS) and Zero Trust Architectures (ZTA) and their integration with emerging machine learning and deep learning methods. It examines how these methods enhance anomaly detection, automated response, and network resilience across different domains including financial services, small enterprises, industrial control systems, and home IoT environments. The review also highlights the role of AI-driven models, response automation, open source tools and federated learning in addressing challenges of scalability, real-time detection, adaptation and computational overhead. It also identifies persistent limitations such as dataset scarcity, model interpretability, and resource constraints. Finally, The review touches on existing challenges and the possible opportunities in their unified adoption, particularly in the form of a AI powered Host-based Intrusion Detection and Prevention System framework.

2.2.1 Unified Threat Management

Siddiqui et al. (2024) explores the development of a security framework designed specifically for home networks. SUTMS framework combines multiple functions, line firewall, IDS (Intrusion Detection System), anti-bot and anomaly detection engines into a single hardware system. Alongside that, it includes flow detection and dynamic access control lists based on Indicators of Compromise. The framework also optimizes performance reducing CPU and memory overhead through modular design and selective IDS signature activation and it analyses traffic flow to identify and block malicious activities without burdening the system resources. The framework supports flexible deployment, including inline or out-of-band setups and supports scalable design integration with existing home networks and IoT ecosystems. The framework features real time detection and blocking of malicious IPs, optional routing, DNS forwarding and DHCP capabilities. Alongside these, the logging engine is compatible with SIEM systems for detailed threat analysis. The framework also comes with a SCMS (Secure Centralized Management System) for device inventory, risk assessment and centralized management which also supports cloud integration for real-time updates and security intelligence. Finally, the findings of the researcher was that the framework could achieve an accuracy of 99.99% in threat detection with 56.83% improvement in memory usage and acceptable CPU usage compared to traditional solutions despite running resource intensive services like IDS/IPS, the SUTMS optimized performance through selective activation and elimination of unnecessary functions by flow detection and NTOP integration [34].

Saad et al. (2019) discusses how artificial intelligence usage specially Bidirectional-Long Short Term Memory (BLSTM) can detect any type of incident in a cloud network format by the use of Unified Threat Management (UTM). The paper makes a comparison of BLSTM against K-Nearest Neighbors (K-NN) to show an accuracy of 98.6% and a mean squared error of 0.002. The authors proposed a machine learning system which includes BLSTM, which will enhance real-time detection of threats and will also provide a prevention mechanism to the problem. Principal Component Analysis (PCA) is used for analysing a 10000 sample dataset from a UTM system, which was also preprocessed and normalised. Optimizers like Adam optimizer and ReLU activation were used so that it can have the ability to detect bidirectional dependencies. It ends with a decision that BLSTM will be better than traditional K-NN for the detection of threats in cloud-based UTM systems as it gets an accuracy of 98.6% whereas K-NN achieves an accuracy of 98.47%. Finally, the authors conclude by raising a commotion that the overall computational cost of the deep learning model can be also considered for optimization in the future [13].

Arifin et al. (2017) presents the implementation of an open source UTM (Unified Threat Management) system. Endian, the UTM system packs features like firewalls, content filtering and intrusion prevention into one package. The UTM was deployed at the core of the network to manage inter-zone traffic like LAN, DMZ and WAN alongside enforcing security policies like port restrictions. A virtualised environment was used to simulate real-world scenarios, which included attack simulations and connectivity tests. The Endian's DHCP service was tested and successfully au-

tomated IP address allocation within the network. Also the system was effective in blocking unwanted contents based on categories, blacklists and organizational policies. Along with these the inter-zone access control was successful in limiting specific ports based on predefined rules. While security testing the UTM's IPS (Intrusion Prevention System) was exposed to threats like port scanning and DDoS attacks, where the Endian could successfully detect some of the attacks but the IPS failed to log certain events due to limited number of snort rules defined. The Endian struggled in advanced threat detection due to limited rule coverage. Finally, the researchers concluded on the Endian UTM to be a solution for small organizations that are looking for low cost solutions for network management[8].

2.2.2 ML and DL in Intrusion Detection

Al-Doori and Alheeti (2025) explores a hybrid approach that enhances Intrusion Detection and Prevention Systems (IDPS) through the integration of AI-based feature optimization with ensemble learning. The authors focus on leveraging Random Forest (RF) combined with metaheuristic algorithms such as Particle Swarm Optimization (PSO), Genetic Algorithm (GA), and Grey Wolves Optimization (GWO) to improve feature selection, reduce class overlap, and optimize model parameters. Using the NSL-KDD dataset, their study evaluates multiple classifiers such as Decision Trees, Random Forest, Neural Network, and Naïve Bayes classifiers across accuracy, precision, recall, and F1-score metrics. Among these, the GWO-RF hybrid model achieved the best performance with 94% in accuracy and f1-score, 95% precision, 93% recall, outperforming baseline Random Forest and other ML models. Their proposed methodology incorporates AI-based feature optimization guided by entropy and information gain, followed by learning and boosting processes for improved classification. Lastly, the potential of deep feature extraction in enhancing intrusion detection is shown with the inclusion of the ResNet-50 with SVM model [37].

Schmitt (2023), in his paper, talks about how the integration of Artificial Intelligence (AI) and Deep Learning (DL) models for the purpose of intrusion and malware detection and the protection of complex digital worlds, which includes cyber-physical systems and IoTs. In the field of cybersecurity, AI can play a significant role in mitigating challenges by enhancing resilience, response, and robustness. Such as Intrusion Detection Systems or IDS, which can be categorized into signature and anomaly-based methods. Although signature-based systems face problems with zero-day attacks (attacks that exploit previously undiscovered vulnerabilities in a system), this system can be boosted using AI-enabled abnormality-based detection systems that detect abnormal patterns, offering robustness. In the case of malware detection, ML algorithms such as random forest, gradient boosting, and DL architectures are used to detect malware in IoT and mobile environments. In the case of intrusion detection tasks, classifiers like gradient boosting and deep learning methods can play significant roles. (Super Learners). Schmitt's study helped to evaluate the performance of AI models across three scenarios, namely network intrusion de-

tection, IoT threat detection, and Android Malware Detection. His work shows that Gradient boosting shows better performance, achieving high accuracy, AUC, and F-scores, and these metrics are further enhanced by super-learners. Addressing challenges related to model interpretability, resilience, and scalability can give us insight into the future of AI-driven cybersecurity [26].

Elsayed et al. (2020) present a hybrid anomaly detection framework in their paper “Network Anomaly Detection Using LSTM Based Autoencoder.” The authors state that the challenges faced by traditional Intrusion Detection Systems (IDS), including their reliance on signature-based methods which are susceptible to zero-day attacks and argue that anomaly detection based IDS empowered by machine learning, provide a better alternative by identifying deviations and abnormalities in network behavior. So the authors propose a deep learning based multi model based method combining Long Short Term Memory (LSTM), Autoencoders and One Class Support Vector Machines (OC-SVM) for greater accuracy and reduced computational costs for detection anomalies in Software-Defined Networking (SDN) environments. The authors introduce a LSTM-Autoencoder, a hybrid model that learns to recreate normal traffic patterns and uses the errors generated during recreation as indications for anomalies. Meanwhile OC-SVM reduces the high dimensionality of the data by applying compressed latent features generated by the LSTM-Autoencoder which also solves the lack of scalability of OC-SVM. Evaluation and performance metrics of the proposed model were done on a dataset containing normal network data and data containing specific attack scenarios such as DoS, DDoS, Botnet and Exploitation. The results highlight the superiority of the LSTM-Autoencoder-OC-SVM model, reporting an F1-score of 0.93 and an accuracy of 90.5% compared to Threshold based Detections F1-Score of 0.825 and accuracy: 74.1%. The Receiver Operating Characteristic (ROC) curve further illustrates the model’s effectiveness by demonstrating its ability in distinguishing between normal and malicious traffic with robust precision. But the authors also acknowledge the limitations of their hybrid model approach. Such as the labeling of malicious traffics based on just reconstruction error thresholds when in reality all traffics behave very similarly. Also there is the fact, the proposed model solely focuses on binary classification which limits its ability to identify attack types. Despite this, the proposed LSTM-Autoencoder-OC-SVM model achieved great results for real time intrusion detection based on its SDN specific environment [16].

Tamy et al. (2020) discuss how to protect industrial systems from intrusion-based attacks through using an intrusion detection system by the use of a hybrid machine learning model. UTM(Unified Threat Management) has been used here with ML for providing layered protection in a system to ensure a global cybersecurity strategy. The focus of the system proposed by the authors is to detect attacks in SCADA-based systems. The authors used a combination of Support Vector Machine (SVM), K-Nearest Neighbors(K-NN) and also Random Forest classifiers, which was later enhanced with PSO, achieving a performance of 99.30% accuracy and it also showed a reduction in Mean Squared Error. All of the studies were conducted on the “Gas Pipeline” dataset, which recreates the environment for SCADA (Supervisory Con-

trol and Data) acquisition environments with a container of multiple attack vectors like Naive Malicious Response (NMRI) and DDOS. The authors came to a decision that when Random Forest is optimised using PSO, it overall showed the best performance with an overall precision of 99.3%, and also showed reduced MSE of .0034. PSO increased the performance of K-NN while decreasing the overall performance of SVM and OneR. The study shows that optimized classifiers project faster model building with Random Forest and K-NN by showing satisfactory results with real-time applications. Overall, the author urged giving importance to the robust security measures in industrial networks. The study concludes that the best solution for IDS at the moment is Random Forest classifier optimised by PSO[17].

Vinaykumar et al. (2019) explores the use of Deep Neural Networks (DNNs) to enhance Intrusion Detection Systems (IDS). This study evaluates both Network-based IDS (NIDS) and Host-based IDS using publicly available datasets to identify optimal configuration for detecting cyberattacks effectively. The proposed framework by the researchers is based on a hybrid detection system that combines NIDS and HIDS for a more capable system in identifying network and host-level anomalies. Alongside that, they have introduced a scalable hybrid intrusion detection system (SHIA) that processes high volumes of data using Apache Spark and GPU-based parallel processing. They have also proposed incorporating advanced NLP-based text representation methods for host-level event classification. To test out and compare the performance against traditional methods, the researchers used well-known datasets such as; KDDCup 99, NSL-KDD, UNSW-NB15 and CICIDS 2017 for NIDS and ADFA-LD for HIDS. DNNs outperformed traditional methods and ML models in terms of accuracy by achieving significantly lower false-positive rates. At the same time, it demonstrated scalability and real-time detection capabilities on large datasets using the SHIA framework. This study highlights the potential of deep learning in IDS to provide a robust and scalable approach in mitigating cybersecurity threats [14].

2.2.3 Host-Based Intrusion Detection Systems (HIDS)

Satilmis et al. (2024) conducted a systematic literature review on many existing Host-Based Intrusion Detection Systems (HIDS) to consolidate research trends, techniques, datasets, and evaluation metrics from 2020 to 2023. Using rigorous filtering across six major databases, the authors find that existing works include a wide range of approaches from classic ML models like Decision Trees, SVM, and Naïve Bayes to advanced DL approaches such as CNNs, LSTMs, and hybrid ensemble models. The review highlights the increasing dominance of anomaly-based HIDS, capable of detecting zero-day attacks through ML/DL profiling, compared to signature-based systems limited to known threats. Commonly used datasets like ADFA-LD, CICIDS2017, NSL-KDD, and newly proposed sets (e.g., DongTing, IoTID20) are evaluated for relevance and balance, noting persistent issues with class imbalance and lack of real-world validation. The study reports that accuracy, precision, recall, false positive rate and F1-score remain the most frequent metrics, with

many studies achieving over 95% accuracy under controlled settings. However, the authors note limitations like dataset obsolescence, insufficient real-time testing, and high computational costs [32].

Letou et al. (2013) presents a comprehensive model aimed at enhancing host security by combining detection and prevention mechanisms. Their model integrates two hierarchical levels of protection: user-level (application space) and kernel-level (system space). Each level uses two detection phases: misuse detection (signature-based) and anomaly detection (behavior-based), to identify both known and unknown threats. They propose using the C4.5 Decision Tree algorithm for misuse detection due to its effective classification and low false alarm rates, and the Support Vector Machine (SVM) for anomaly detection given its robustness in binary classification. Evaluation of the architecture show that this hybrid model effectively detects intrusion types such as DoS, Probe, R2L, and U2R while maintaining system performance. The paper highlights that combining SVM and C4.5 enables high detection accuracy (up to 99.82%) while minimizing false positives. However, the authors state challenges remain regarding delay in alert generation, host resource consumption, and the need for real-time adaptability [3].

2.2.4 Surveys, Benchmarks, and Dataset Realism

Mankar et al. (2024) explore ways to combat cyber threats like malware and phishing attacks, which result in huge economic losses worldwide, and highlight the crucial part that computerized harmful URL identification plays. It examines the outcome of many machine learning models, emphasizing the efficiency of ensemble methods such as Random Forest and Extra Trees, which detected dangerous URLs with an accuracy of over 91%. These models function better than more straightforward techniques like AdaBoost which has 82.01% accuracy and Naive Bayes which has 78.95% accuracy. Decision Trees is 90.96%, K-Nearest Neighbors is 88.96%, and Stochastic Gradient Descent (81.28%) are among the various other models that were assessed. The investigation shows that the capacity of tree-based ensemble models to manage interaction among features and minimize overfitting makes them excellent. However, problems like class disparity still exist since minority dangerous URL categories like malware and scamming have poorer accuracy and recall than most benign URLs. To increase the detection rate, resistance to undetected exploit attacks, and expansion, the research also suggests potential routes that call for more sophisticated deep learning techniques like BERT and LSTM. These methods might improve the resilience of cybersecurity frameworks and assist in addressing the dynamic nature of cyber threats [30].

Podder et al. (2021) reviews the usage of deep learning (DL) models in the field of cyber security, in comparison to traditional machine learning models (ML). The authors provided the accuracy and precision metrics of various DL models (DNN, RBM, CNN, etc) used on a wide variety of datasets that were used on other research

papers and noted that RBMs and Autoencoders were the most utilized DL methods counting for about 50% of use cases in the reviewed research papers, as well as RNNs which are popular for its ability to combat a diverse category of cyber threats. They conclude that RBMs and LSTMs provide the highest accuracies upto 99.72% and 99.8% respectively on both custom and standard of datasets. The authors then discuss the issues that obstruct the mainstream adoption of deep learning methods in the cybersecurity industry, ranging from taking too much time to analyze data, current systems focusing on combating only one type of threat, the scarcity of standard datasets, to DL tools inability to provide the cause of errors. To solve these issues, the authors suggested that further research should be focused on developing tools combining multiple DL approaches to correctly detect and prevent a greater range of attack vectors to combat the largest spectrum of cyber security challenges [19].

Buczak et al. (2015) present an in-depth study of the data mining (DM) and machine learning (ML) approaches used in intrusion detection systems (IDS). It identifies the convenience and drawbacks of IDS systems by classifying them into misuse-based, anomaly-based, and hybrid approaches. Using pre-established signatures, misuse-based approaches successfully identify existing invasions but have trouble identifying unexpected exploits. On the other hand, anomaly-based techniques, which simulate typical activity and spot discrepancies, are excellent in spotting unidentified assaults but have a significant incidence of false alarms. The goal of hybrid systems is to integrate the advantages of both of them. The researchers look at supervised, unsupervised, and semi-supervised ML/DM methods, such as genetic algorithms, Bayesian networks, clustering, decision trees, artificial neural networks (ANNs), and ensemble learning. The study evaluates each approach according to its applicability for different IDS tasks and its computational demands. Random Forests are excellent at managing unbalanced datasets and minimizing overfitting, whereas decision trees are commended for their quick categorization and easy rule derivation. With a false alarm rate of 0.9%, supervised learning techniques like decision trees were able to identify malicious domains with up to 98% accuracy. According to the intensity of the assault, collaborative methods such as Random Forests produced detection accuracies ranging from 87% to 95% with minimal false-positive ratios. Although neural networks have difficulties with computational complexity, they claim a success rate of about 93% in detecting abuse. Although the proportion of false alarms was occasionally high (such as 13%), unsupervised clustering techniques showed up to 80% detection accuracy for unexpected assaults, indicating space for reformation in anomaly-based systems. The paper highlights how popular datasets like DARPA and KDD 1999 have narrowness because of antiquated attack techniques that limit the potential for generalization of the models. It offers fundamental insights on lowering the number of false positives and enhancing IDS scalability by highlighting the significance of feature selection, dimensionality reduction, and customized approaches for certain scenarios [5].

2.2.5 SIEM and XDR for Scalable Threat Detection

Ali et al. (2025) presents a comprehensive framework that integrates machine learning (ML) with Security Information and Event Management systems to improve the detection and response to advanced persistent threats (APT). The authors identify key challenges faced are high false positive rates and excessive manual intervention during incident response. To address these, they design six event detection modules that monitor network traffic, system behavior, user activity, file operations, malicious IPs, and domains using Splunk. The proposed framework builds on an existing APT dataset by introducing behavioral indicators and new alert types. Then Random Forest and XGBoost classifiers are applied to distinguish between complete and incomplete APT attack patterns. The proposed model achieves 99.6% accuracy compared to existing SIEM approaches. Algorithms for alert correlation, event detection, and attack classification are showcased to show how supervised learning can automate the identification of various APT stages. The results show that the introduced AI-driven approach can effectively optimize incident response times and reduce analyst workload [36].

Manzoor et al. (2024) investigate the effectiveness and capabilities of the open source SIEM systems for SMEs (Small and Medium Enterprises) facing financial constraints. The researchers study and evaluate open source SIEM systems such as Wazuh, SIEMonster, OSSIM and Elastic Security in terms of features, scalability and performance under high traffic conditions. They have designed a ranking system for the SIEMs, assessing primary features like correlation rules, log integrity and fault tolerance and a few secondary features such as scalability and community support. Then they tested the SIEMs in a simulated SME network with devices such as firewalls and intrusion detection systems and the evaluation was based on EPS (Events per Second) processing, scalability and security feature coverage during simulated DoS and malware attacks. In the tests, Wazuh achieved the highest overall score with 47/50 excelling in EPS and advanced feature implementation, including rule customization and log retention. Next was Elastic security with 42/50 which excelled in user authentication and visualization. SIEMonster was next in line with 37/50 which lacked rule customization capability. Lagging behind in the last place was OSSIM with 26/50 due to limited correlation rules and basic log encryption. Wazuh can be highlighted to be the best option thanks to its advanced features, scalability and ease of integration. The researchers conclude that open source SIEMs are a viable solution for the SMEs but they require better documentation for ease of use and extended features [31].

Pissandis and Demertzis (2023) in their paper showcase the integration of Artificial Intelligence (AI) and Machine Learning (ML) within frameworks associated with cybersecurity. This paper emphasizes on Open XDR, detailing how it can play a crucial role in cybersecurity with the help of ML and AI. This study highlights how Open XDR strengthens cybersecurity by combining IDS, SIEM, EDR, etc. This paper also emphasizes on how AI-powered SIEM enhanced data analysis which prioritizes alerts and helps in the reduction of false positives, which is a feature applicable to blockchain-based ledgers. Here, we can also see the challenges

AL/ML integration can face such as privacy, ethics, and skill limitations which play a crucial role in in implementing AI/ML-driven UTM systems [25].

2.2.6 Fintech Threat Landscape

Biplob et al. (2024) explores the scope of machine learning (ML) and deep learning (DL) in enhancing cybersecurity within Bangladesh’s rapidly growing digital financial infrastructure. The paper discusses challenges posed by cyber threats and the potentials of advanced AI-based technologies to address the vulnerabilities. Legacy systems and rapid adoption of mobile banking leaves financial institutions open to threats like malware, phishing and other technological vulnerabilities. The solutions the researchers suggested contain AI algorithms to analyze vast amounts of data to detect any anomalies in real time to detect and prevent cyberattacks. Human errors and fraud detection can also be done through using previous transactional data to recognize transaction patterns. Finally through automation, the cost and expertise needed to deploy security infrastructures can be lowered and streamlined while ensuring the security meets regulatory standards. There are challenges to implement these solutions as well which the authors have pointed out. The lack of large and high quality datasets in Bangladesh, mean that the success rate of ML/DL models will not be satisfactory. Alongside that, most of these institutes lack in computational and storage capacity which is a huge drawback while trying to implement AI based solutions. Finally, the lack of trained professionals in the field of data science and cybersecurity will be another hurdle to tackle. The final recommendations from the researchers were to raise awareness among everyone about cyber risks and the best practices to avoid them, slowly incorporating AI, blockchain and behavioral analytics using ML/DL to strengthen defense mechanisms gradually alongside creating a regulatory body to enforce and enhance cybersecurity guidelines [27].

Hossain et al. (2023) explores the challenges and solutions in securing FinTech systems in Bangladesh against cyber attacks. The authors have mentioned the vulnerability of the FinTech sector as the attackers disproportionately target MFS (Mobile Financial Services) specially due to poor security measures. They have also found ransomware (46%) and social engineering (29%) to be the most prevalent attack methods. And in their FinSec framework they have proposed a solution in three units. Them being; action, knowledge and simulation unit. In the Action unit, the research team have emphasised on the human factor alongside improving digital and physical infrastructure. With solutions such as, strengthening cyber security knowledge of the end user through gamification to combat threats social engineering, 3WA (3 way authentication) to enhance security measures along with enhanced training for the merchants and securing the physical factors using technologies such as; NGFW (Next-Generation Firewall), NGIPS (Next-Generation Intrusion Prevention System). The knowledge unit consists of the part of potential threat study on the pattern of different cyber attacks and with the outcome of the study the research team suggests implementing upgrades and updates on the security system. Finally in the simulation unit, the financial organizations are encouraged to have

a dedicated team of cyber security experts to do forensic analysis on attacks and alongside that doing penetration tests on the security systems while maintaining the physical infrastructure and keeping the security system under constant evaluation [22].

Nicholls et al. (2021) present an extensive survey of deep learning and machine learning approaches to intercept financial cybercrime ecosystems such as financial fraud, hacking, and social engineering for unscriptural financial profit. The study allocated the financial cybercrime ecosystem from some perspectives, like the various fraud techniques used by criminals, system and algorithm limitations. To oppose these risks, the authors stress the growing dependence on those sophisticated machine learning and deep learning models including Graph neural network(GNNs), Anomaly Detection(AD) and Hybrid approaches. Also highlighted that Group anomaly detection (GAD) is more capable than anomaly detection (AD) for identifying clusters of deceptive actions. The efficient data encoding neural network model such as denoising Encoder and the sufficiency of Graph Neural Network (GNNs) for revealing deceptive linkages in financial networks are some of the key outcomes. Moreover, some barriers were explored, such as the scarcity of leveled datasets, scalability problems with dynamic data and stringent criteria for explainable AI to view transparency and regulatory compliance. To sum up, the authors discuss it as a fundamental resource for comprehending how financial cybercrime is changing and how deep learning (DL) might help to figure it out for resolving it. It emphasizes how decisive it is to assemble domain knowledge with cutting-edge computational methods to vow feasible, effective, and rigid fraud detection systems [18].

Hossain et al. (2017) analysed how financial organisations are targeted for different kinds of cyberattacks through the datasets that were available on the surface web(Paysim, Banksim) and the deep web (SOCRadar). The research shows a total of 71 types of common attacks in the cyber world where the study concludes phishing is the most popular technique that is used by cybercriminals. Through analysing the databases it is understood how through exploitation of vulnerabilities to overall steal vital information, financial breaches and assets. The overall paper also evaluates how such fraudulent activities function based on their location, gender and transaction method. The study shows that women are mostly targeted as victims of financial cyber attacks. The paper make note of this problem and suggest a four-layer security framework called Finframe. This framework has a total of four components which are human layer, application layer, transport layer and physical layer. Each layer has a different set of roles. The research encourages the use of multi-layered security solutions to protect the financial transactions of any business or a normal commute so that they can be safe from cyberthreat [23].

2.2.7 Real-Time DDoS Detection and Feature Engineering

Goldschmidt and Kučera (2024) discuss about a feature-extraction method called Windower which can be used for real-time detection of DDoS attacks. This system integrates two currently and widely used flow-based and packet-based detection systems into one package which mitigates the disadvantages of these two systems significantly. This system first processes network packages by extracting values like destination and source IP addresses, ports, protocol types, etc. This system by default excludes payloads which make it independent of encryption and application-layer specifics. These packet features are combined into a time-based sliding window mechanism based on source IP. Each window computes intra-window statistics (Packet counts, sizes, protocol shares, arrival times). Then, additional inter-window statistics are calculated which captures communication patterns over time. This enables the system to distinguish between normal traffic and abnormal traffic patterns which indicate a DDoS attack. Then the aggregated features are fed into KilNet. This detects the abnormalities by finding out the value of the Root Mean Square Error (RMSE). A predefined threshold is set early and the value from the RMSE is compared against that value. If a DDoS attack is detected, mitigation is triggered which blocks traffic from the sources launching the attacks using Access Control Lists. This system has a series of advantages which include high detection accuracy with 99 percent of mitigation against flood attacks and 92 percent with slow attacks with false positives around one percent. Other benefits are that it works in real-time, and has low computational overhead, flexibility, and resilience. Although it has many advantages, it has its flaws such as it is vulnerable to IP randomization, delayed detection for low traffic, high memory consumption, and limited context analysis [29].

2.2.8 Vulnerability Scanning and Low-Cost Security

Seara and Serrão (2024) discussed how system vulnerabilities can be the reason of the escalation of cyber-attacks faced by organizations which result in financial loss and stoppage in operation, so they proposed a vulnerability scanner which will be open-sourced and it can automate security audits and it will provide a low-cost solution but also scalable at the same time. The system that the authors proposed has two important software components: they are agents which will be used for scanning and discovering any network by the use of libraries of python like Scapy and for vulnerable detection python3-nmap, another one is a server which will be responsible for the facilitation for the communication through rest API, storing results for databases and it will also handle various case of reports using report dissemination. To validate the overall system built by the authors they provided some real-world evaluation by the use of some testers. This includes reliability, accuracy and usability of the system. As the tool can stay afloat without any interruptions even in heavy resource-based systems, and it was able to detect known cases of vulnerabilities that were thrown at it. An emphasis on the need for a graphical user interface was expressed as the system didn't have any. Also, the system needs to be tested in an even bigger scale testing environment. For future improvements, the authors expressed desire to do threat analysis and remediation by the ML and AI integration,

and to build an even better reporting capability for the system [33].

2.2.9 Automated Response and Reinforcement Learning

Ismail et al. (2023) examines how Artificial Intelligence can be applied to automate incident response in cybersecurity, improving detection speed, accuracy, and adaptability. Their study proposes an AI-driven architecture composed of data collection, threat intelligence, anomaly detection, and automated orchestration layers, all of which are aimed at streamlining the detection and response loop through supervised and unsupervised learning models such as SVM, Gradient Boosting, Decision Trees, K-means Clustering and Isolation Forest. The authors evaluated their approach based on accuracy, precision, recall. Their results show Gradient Boosting achieving the highest accuracy 96.5%, precision 98.4%, recall 94.1%, followed by SVM with strong yet slightly lower results. Unsupervised methods like Isolation Forest and K-means, showed potential for detecting unknown anomalies with less accuracy. Furthermore, Reinforcement Learning (RL) was integrated to automate response decisions in real time, reducing average response time to 1.5 seconds compared to rule-based systems. The paper also discusses ethical considerations such as emphasizing fairness, transparency, and GDPR compliance. Overall, this research highlights that AI-driven automated incident response frameworks significantly enhance threat detection efficiency and resilience, and ethical governance in AI-powered cybersecurity systems [24].

2.2.10 Generative AI in Cybersecurity

Ferrag et al. (2025) states how modern generative AI can play an important role in the world of cybersecurity. Modern LLMs such as ChatGPT, LLaMA, and Gemini have the capability of completely revolutionizing this particular field as these can automate complex tasks along with complex decision-making and can significantly amplify and overhaul defense mechanisms. Ferrag et al. (2025) highlight there are nine key applications in the world of cybersecurity which include threat-detection analysis, security automation, phishing detection, cyber forensics, immediate response to incidents, usage of advanced chatbots, training, penetration testing and security protocol analysis and verification. Although LLMs bring a ton of benefits in this field, there are still some issues that need to be addressed. Threats such as prompt injection attacks, data poisoning, adversarial inputs, DoS attacks, and insecure output handling can sabotage systems putting the entire security at risk. LLMs depend on high-quality datasets which are also domain-specific but existing datasets lack diversity and size which questions model robustness. In order to enhance performance and security, advanced techniques such as Retrieval-Augmented Generation, Reinforcement Learning with Human Feedback, etc are being employed to enhance performance and security. Challenges like adversarial attacks, data quality issues, computational demands, limitations, etc are some roadblocks that LLMs face in order to solve cybersecurity problems but as time progresses, it is expected

that LLMs with the above-mentioned techniques will make the internet more secure for its users [38].

2.2.11 Open-Source and Rapid Security Prototyping

Jones et al. (2007) in their paper discussed a method called Patchwork Prototyping. The main goals of Patchwork prototyping include capturing accurate requirements for software development, combining iterations with low-cost and fast iterations with highly functional prototypes, and providing horizontally broad and vertically deep prototypes. How this process works is by first, making a calculative prediction based on the sought design, then selecting open-source tools supporting special functions, integrating these tools into a functional prototype, deploying the tools to get feedback, and then finally reflecting on the prototypes to make further improvements on the next iterations. Two case studies further solidify these claims: Project Alpha and Project Beta. Project Alpha is a collaborative tool for environment engineers that uses OSS, such as MediaWiki. Project Beta on the other hand built tools for inquiry-based learning, starting from the beginning and later adding OSS such as phpBB. There might be a lot of benefits to this approach but there are certain limitations such as it requires skilled developers familiar with OSS and its integration [2].

2.3 Summary of Reviewed Researches

The advancements in the cybersecurity domain highlighted in the literature review gives credence to the proposed framework Avantgarde. The application of AI models such as Ensemble learning, LSTM, Autoencoders, RNN in anomaly detection and other avenues of cyber security have demonstrated the ability of AI in such cases along with displaying advancements in real time threat detection as well as reduction of false positives. The reviewed literature on Host-based Intrusion Detection and Prevention Systems (HIDPS) highlights the growing importance of integrating machine learning models for host-level anomaly detection and active defense. Studies have demonstrate that hybrid models combining signature-based and behavior-based mechanisms, supported by classifiers like SVM, Decision Tree, and other ensemble methods can achieve high accuracy levels, effectively identifying threats such as DoS, R2L, and U2R while maintaining system efficiency [3][32]. The studies focusing on HIDPS emphasize lightweight, adaptive, and explainable architectures that can scale across host environments, offering real-time detection and response capabilities critical for dynamic network ecosystems. The reviewed papers also highlight the UTM and SIEM systems efficiency, robustness, log monitoring and real time analytics. Zero Trust architecture has also proven effective in enforcing concurrent verification, tailored authentication policies and minimal access control which all contribute to minimizing intrusion or malicious access in the network. Despite the advantages showed by the aforementioned technologies, researchers across the board, state notable challenges in the implementation of AI driven IDPS frameworks

such as the AI module being constrained by dataset limitations, adversarial AI attacks, lack of explainability of anomalies due to the black-box effect of models and high computation costs. Furthermore, researches on AI-driven automated incident response underlines the transition from reactive defense to proactive response and mitigation [36] [24]. By integrating supervised and reinforcement learning models with automated response mechanisms, these studies achieve reduced response times and improved resilience of the overall system, as well as including methods for alert prioritization, event correlation, and autonomous containment which further validates the feasibility of real-time, self-adaptive defense frameworks that align with our research objectives. Thus, all reviewed literature play important roles as inspirations, affirmations and guidelines in the design and development of Avantgarde and its ability to provide optimal security despite all the challenges.

Chapter 3

Requirements, Impacts and Constraints

3.1 Final Specifications and Requirements

The framework needs a modular architecture which can integrate outputs from both its native Host-based Intrusion detection and prevention system components and an independent intrusion detection system, a suitable logging method or database for storing logs and decisions and lastly efficient real time traffic capture parameters. The framework will be heavily relying on live traffic data and processing it, which must include normalisation, encoding and feature reduction steps in order to optimize overall machine learning performance. Multiple machine learning models are intended to be trained and tested, all of which are to be evaluated with the use of metrics such as accuracy, precision, recall and false-positive rates. Several datasets such as the CAIDA Anonymized Internet Traces Dataset, CIC-IDS2017, UNSW-NB15 and BCCC-CIC-IDS2017 are selected and tested to choose the best one which should have comprehensive labeling and a variety of attack types. The framework can be deployed and tested on linux environments which can ensure more safety and freedom of access in network simulation environments.

3.2 Societal Impact

The framework will provide enhanced security for individual computing systems by offering robust host-based protection against various cyber threats by implementing a hybrid intrusion detection and prevention system at the endpoint level. The automated response and accessible nature of the framework will help individual users and small businesses improve their cybersecurity posture without requiring extensive technical expertise or complex network-level security infrastructure. Despite the framework operating at the individual host level, widespread adoption could contribute to overall network security of an area by reducing the number of compromised endpoints that could potentially be used in larger-scale attacks or botnets.

3.3 Environmental Impact

The framework’s environmental impact is expected to be minimal as it runs on existing hardware and eliminates the need for additional physical security appliances, thereby reducing electronic waste as well as purchasing and maintaining dedicated security hardware. The overall environmental footprint remains computationally low as it runs on each individual endpoint without needing any cloud based services or large scale servers. The open-source nature of the independent intrusion detection system and the framework’s compatibility with Linux environments promote longer device lifecycles by providing security updates and protection for existing hardware. The framework thus supports sustainable computing practices by maximizing the security potential of existing infrastructure.

3.4 Ethical Issues

In today’s digital landscape, users regularly share personal and sensitive information online, making ethical data handling essential for any digital service. The proposed framework addresses these concerns by operating solely within the host environment using flow level network traffic analysis for intrusion detection and prevention. The system adheres to privacy by design principles by collecting only the minimum network traffic data necessary for security functions and avoids deep packet inspection which can access information in the payload data in IP packets. Since all processing occurs locally on the individual host, no user data is transmitted to external servers or third parties, maintaining user privacy and data sovereignty. All security decisions and logs also remain on the local system, giving users complete control over their security data telemetry, balancing effective cybersecurity with respect for user privacy and data protection rights.

3.5 Project Management Plan

The project began with comprehensive research on existing threat management systems comprising both network and host-based security approaches to establish the technical foundation by gathering as much information as possible. The research scope was then narrowed down to focus specifically on hybrid intrusion detection and prevention systems, followed by systematic dataset collection and evaluation from available sources including CAIDA, CIC-IDS2017, UNSW-NB15, and BCCC-CIC-IDS2017 datasets.

Framework architecture design and ML model training using the selected datasets were done parallelly with the team splitting into two groups focusing on respective tasks. After several iterations, a working framework architecture was developed with specific components and their purpose. Then, prototype development research was conducted to understand implementation requirements for the proposed framework. Meanwhile, after testing with the aforementioned datasets, the BCCC-CIC-IDS2017 dataset was determined to be the most suitable for the proposed framework’s machine learning module due to its comprehensive labeling and diverse attack type coverage.

Lastly, the prototype development and model training and testing phase began. The

team as previous was organized into two specialized groups: one focusing on ML model training, evaluation and analysis, while the other concentrated on prototype development primarily on Linux environment deployment with support for windows. Prototype development proceeded with real-time traffic capture, optimizing traffic data with normalization, encoding, and feature reduction methods for processing in ml models. Following completion of all components, the trained models and prototype were integrated to showcase the hybrid Host-based Intrusion Detection and Prevention System framework.

3.6 Risk Management

Several project risks need to be considered for developing mitigation strategies for the proposed framework. One of the biggest risks is that sophisticated Attacks can bypass both ML based anomaly detection and External IDS tools signature-based detection, gaining unauthorized access to the host, this can be mitigated using Zero Trust access controls such as continuously verifying the identity of the host incoming packets. But for normal cases, the layered defense with using both anomaly detection and signature-based detection and validating each other's output also ensures that even if one detection mechanism fails, the other can detect because of the different way each identifies threats. A high volume of network traffic could overload the entire system, potentially affecting performance and leading to slower response times or missed detection of threats. BPF filters or DNS sinkholes are used to reduce traffic noise and focus only on relevant traffic. Then there is the risk of packet drops, capturing a very high amount of packets every second can overwhelm the packet capture module, methods such as batch processing, time buffer can be used to prevent this. Performance degradation of the entire system because of running multiple models concurrently, to avoid this model parallelization and load balancing can be used. Using the databases inbuilt integrity checks to avoid risks such as tampered logs. The decision mechanism could mistakenly classify legitimate network traffic as malicious, leading to service disruptions or user access denial. To manage this, there can be an unblocking override function in the admin panel to unblock blocked IP's.

3.7 Economic Analysis

The economic benefits of the proposed framework are significant for all hosts, as it offers an affordable, effective solution to host-based intrusion detection and prevention without requiring expensive security infrastructure or extensive technical expertise. By leveraging existing hardware, the framework minimizes both upfront costs and long-term maintenance expenses. Because of the automated response capabilities of the framework, there is no need to have dedicated personnel for making decisions on detected threats. Additionally, the use of open source independent intrusion detection system tools eliminates the need for licensing fees and promotes cost-effective scalability. Initial development and deployment costs are also negligible as building prototype and training machine learning models do not need extensive computational power or high end computers. Lastly for the consumers, the overall cost efficiency is quantitatively increased with the reduction of potential threats.

Chapter 4

Proposed Methodology

4.1 Methodology Overview

The main purpose of the *Avantgarde* framework is to capture network traffic flows in the network, pre-process the flows and then generate model predictions, cross match the predictions against the alerts from a independent IDS alerts, generate an appropriate decision and response automatically. During the study of other research papers, we noticed that while many used one or two sets of the discussed features or technologies, none attempted to integrate all such technologies into a framework, possibly due to integration overheads, resource consumption, data consistency, error handling, and research interest. Despite these challenges, the implementation of multiple technologies into a single framework can have several benefits such as real-time traffic analysis, centralized monitoring and logging, streamlined incident response, threat analytics. Therefore, we attempt to introduce a HIDPS framework called *Avantgarde* that can leverage all the discussed benefits while reducing the stated challenges. The following figure 4.1 showcases the proposed architecture of *Avantgarde* in detail:

Avantgarde Architecture Diagram:

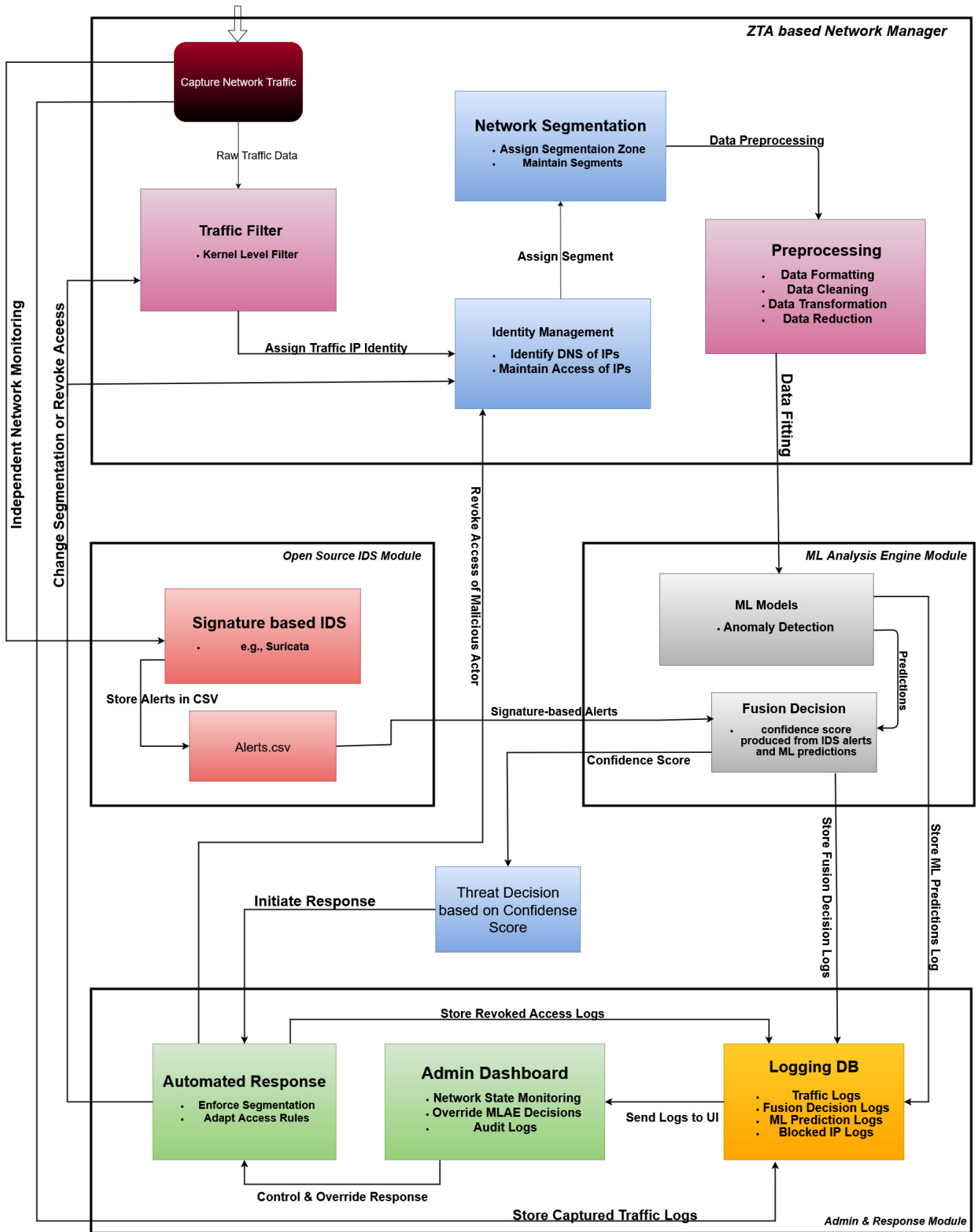


Figure 4.1: Avantgarde architecture

4.2 Preliminary Design Process

Our proposed framework is a Host-based Intrusion Detection and Prevention System that utilizes both ML models and other tools to detect and mitigate security incidents by analyzing network traffic. In order to gain the benefits of incorporating the main feature categories of machine learning, zero-trust architecture, automated response and open source ids, and their corresponding components as per the architecture diagram, we have decided to use a structured validation approach which can be broken down into the IDS section and the IPS scenarios:

IDS Scenario: The IDS scenario comprises of machine learning based anomaly detection validated with traditional signature based IDS output in a hybrid detection pipeline to calculate confidence score. Network traffic, once filtered and segmented, is inspected simultaneously by the Machine Learning Analysis Engine (MLAE) and an independent IDS like *Suricata* or *Snort*. While the signature based IDS applies known rules and signatures to identify and log malicious packets, the MLAE depends on its training to find anomalies in packet flow behavior. Their outputs are cross-validated with a confidence-driven decision method, mapping ML and IDS tool outputs into a continuous score that reflects certainty. This produces a single canonical verdict with a confidence score that reduces both false positives and false negatives as well as forming the basis for downstream decisions. The entire flow (Packet Data, Score, Decision) is logged back into the logging db for human oversight.

IPS Scenario: In the IPS scenario, the validated threat detections from IDS scenario triggers automated response actions. Once threats are confirmed, the response function integrates IDS results with preset access policies to override current access by blocking of hostile IPs, revoking segmentation access in line with zero-trust architecture principles. Automated response functions adapt firewall and filter rules on the go, though blocked IPs can be rolled back to safe states through human oversight. To reduce overhead, response mechanisms are only triggered when threats are detected, and all responses are recorded in the logging db for auditing. Over time, these stored decisions and configurations provide both a forensic trail and a source of explainability of the MLAE output, closing the loop between detection, response, and so enforcing network security.

4.3 Component Functionality

With the IDS and IPS scenarios outlined to detail the operational flow of threat detection and automated response, we will describe the core components that enable these functionalities within each feature category. The following sections provide a structured breakdown of the components across Network Data Handling, Zero-Trust Architecture, Machine Learning Analysis Engine, Logging Database, Automated Response, and Independent IDS modules as illustrated in the architecture diagram.

Network Data Handling:

- **Traffic Capture:** Real-time packet capture with Scapy BPF filters to exclude noise and forward it to the pre-processing module.
- **Pre-processing:** The raw network traffic data goes through several pre-process steps such as data formatting, cleaning, transformation, and reduction to prepare for model feeding.
- **Independent IDS Stream:** Independent IDS like Suricata generate alerts which with packet details are written to a suitable file format like CSV, flushed in short intervals for batch analysis.

Machine Learning Module:

- **ML Analysis Engine (MLAE):** Processes pre-processed network traffic flows and applies multiple trained ML models to generate predictions. These predictions identify whether a flow is benign or malicious, and classify the type of attack if detected.
- **Pattern Validation:** Cross-checks MLAE predictions against independent IDS logs (e.g., Suricata) to validate anomalies and reduce false positives. This ensures that both statistical/ML-based patterns and signature-based evidence are considered before a verdict is reached.
- **Confidence-Based Decision:** A confidence-driven decision process maps external IDS signals and ML model outputs into a unified confidence score. This score is smoothed over time and then converted into actionable decisions such as BLOCK, ALLOW.

Zero-Trust Architecture:

- **Domain Verification:** Reverse DNS lookups and trusted suffix checks automatically authorize safe domains.
- **Identity Management:** Allocates IPs to appropriate segmentations in the network based on Domain Verification. Tracks each active/known connection with status.
- **Segmentation and Least Privilege:** Divide the network into isolated access zones based on ports, traffic pattern or resource needs. Active connections are isolated logically; revoked hosts lose all privileges instantly.

- **Traffic Control:** Regulate and inspect all traffic within and between network segments to ensure it adheres to established security policies, blocking unauthorized or suspicious activity.
- **Continuous Monitoring:** Maintain real-time visibility of all network activities and access requests. Adaptive enforcement is ensured at runtime by making sure that the logging of all the access and packet decisions is done.

Automated Response:

- **Decision Enforcement:** On the basis of the confidence score decision is given through firewall rules whether or not to block or unblock IPs dynamically.
- **Access Overrides:** Automated revocation or restoration of segmentation/access rules under zero-trust enforcement.
- **Audit Logging:** All actions (block, allow, rollback) recorded in DB for accountability and retraining.

Independent IDS:

- **Real-Time Analysis:** Analyses network data, ensures reinforcement of primary level threat response and detection of the framework by working as a secondary source.
- **Independent Signature Logs:** Storage of the logs are done by ensuring to keep up with pattern validation and forensic tracing of decision process from signature based IDS perspective.

4.4 Implementation of Framework

To test the efficiency of the proposed architecture, a prototype was built. The prototype was built in Flask, a micro web framework built in Python. Flask was chosen for its versatility and the variety of important Python libraries such as Scapy, scikit-learn, pandas etc. Based on the roles played by various features of the prototype, the prototype can be considered comprising of four interconnected subsystems:

- **Network Data Processing:** Captures live traffic, filters network noise with Scapy BPF rules, and prepares packet data for MLAE.
- **Segmented Access and Management:** Implements Zero-Trust principles by tracking identities, enforcing least-privilege access, and dynamically blocking or authorizing hosts.
- **Machine Learning Analysis Engine:** Applies anomaly detection models to preprocessed traffic and outputs predictive labels for threat validation.

- **Independent signature IDS:** Runs Suricata as an independent stream, the produced signature-based logs are cross-checked and validated with ML predictions.



Figure 4.2: Avantgarde Prototype Admin Dashboard

The Figure 4.2 showcases the prototypes dashboard which provides a real-time overview of active connections, connection status based on segmentation, packets count, and per-IP traffic visualization within the monitored hosts network.

4.4.1 Network Data Processing Overview

Network Data Processing is first done by capturing live traffic with Scapy from the monitored network environment. Irrelevant or noisy packets are filtered out using Berkeley Packet Filter (BPF) parameter in Scapy, and then preparing the filtered out data for further analysis. This pre-processing ensures that only relevant flows are passed to the Machine Learning Analysis Engine, reducing computational overhead and minimizing the risk of false alarms caused by background noise or internal traffic of the prototype. By processing packet flows into consistent feature representations, this stage of the prototype forms the framework’s data pipeline.


```

Terminal
_voting': 'Malicious', 'RXC_voting': 'Benign'}
[MLAE] 192.168.0.111:51042 -> 57.144.140.141:443 | {'RX_stacking': 'Benign', 'RX_voting': 'Benign', 'RXC_voting': 'Benign'}
[MLAE] 57.144.140.141:443 -> 192.168.0.111:51042 | {'RX_stacking': 'Benign', 'RX_voting': 'Benign', 'RXC_voting': 'Benign'}
[MLAE] 192.168.0.111:34691 -> 155.133.225.21:443 | {'RX_stacking': 'Benign', 'RX_voting': 'Benign', 'RXC_voting': 'Benign'}
[MLAE] 192.168.0.106:27036 -> 192.168.0.255:27036 | {'RX_stacking': 'Malicious', 'RX_voting': 'Malicious', 'RXC_voting': 'Malicious'}
[MLAE] 155.133.225.21:443 -> 192.168.0.111:34691 | {'RX_stacking': 'Benign', 'RX_voting': 'Malicious', 'RXC_voting': 'Benign'}
[MLAE] 192.168.0.111:56936 -> 57.144.140.145:443 | {'RX_stacking': 'Benign', 'RX_voting': 'Benign', 'RXC_voting': 'Benign'}
[MLAE] 57.144.140.145:443 -> 192.168.0.111:56936 | {'RX_stacking': 'Benign', 'RX_voting': 'Benign', 'RXC_voting': 'Benign'}
[MLAE] 192.168.0.111:56936 -> 57.144.140.145:443 | {'RX_stacking': 'Benign', 'RX_voting': 'Benign', 'RXC_voting': 'Benign'}
[MLAE] 57.144.140.145:443 -> 192.168.0.111:56936 | {'RX_stacking': 'Benign', 'RX_voting': 'Benign', 'RXC_voting': 'Benign'}
[MLAE] 192.168.0.111:58964 -> 104.16.65.50:443 | {'RX_stacking': 'Benign', 'RX_voting': 'Benign', 'RXC_voting': 'Benign'}
[MLAE] 104.16.65.50:443 -> 192.168.0.111:58964 | {'RX_stacking': 'Benign', 'RX_voting': 'Benign', 'RXC_voting': 'Benign'}

```

Figure 4.3: Packet Capturing with Scapy and Model Predictions

The Figure 4.3 shows the prototype capturing packets, then the processing and model feeding is done to captured traffic and the output of each ML model is shown.

4.4.2 Segmented Access and Management Overview

The Segmented Access and Management enforces Zero-Trust principles by continuously tracking device identities and maintaining strict least-privilege access policies. Each IP is dynamically authorized, blocked, or labeled unknown, depending on its security posture, with decisions being updated in real-time based on decisions resulting in the confidence score given by the Machine Learning Analysis Engine. This ensures that even if a trusted IP exhibits abnormal behavior, its access can be immediately revoked, reducing lateral movement risks and enhancing the resilience of the network.

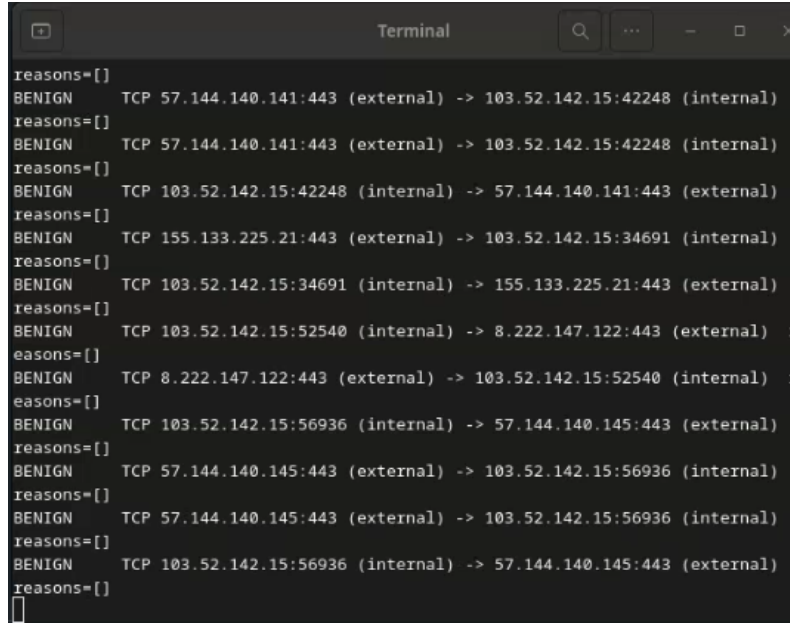
47.236.244.181	-	47.236.244.181	unknown	02.11.37, 07.10.2025	02.11.47, 07.10.2025	Authorize Revoke
57.144.140.145	-	edge-dgw-shv-01-ccu2.facebook.com	authorized	02.11.33, 07.10.2025	02.11.44, 07.10.2025	Authorize Revoke
8.222.147.122	-	8.222.147.122	unknown	02.11.32, 07.10.2025	02.11.47, 07.10.2025	Authorize Revoke
155.133.225.21	-	155.133.225.21	unknown	02.11.31, 07.10.2025	02.11.46, 07.10.2025	Authorize Revoke
57.144.140.141	-	edge-star-shv-01-ccu2.facebook.com	authorized	02.11.28, 07.10.2025	02.11.44, 07.10.2025	Authorize Revoke

Figure 4.4: Segmentations status of connections

The Figure 4.4 shows the segmentized state of various IPs in "authorized", "unknown", as well manual override option for human supervision.

4.4.3 Independent Signature IDS Overview

Suricata was chosen as the independent Signature based IDS during implementation of the prototype to leverage the precision of known threat signatures. It ran in parallel as a separate detection stream, generating signature-based alerts with deterministic reasons for detecting threats. These alerts are fetched from the eve.json file and are saved into a CSV file so the prototype can fetch the alerts and process them in the Machine Learning Analysis Engine.



```
reasons=[]  
BENIGN    TCP 57.144.140.141:443 (external) -> 103.52.142.15:42248 (internal)  
reasons=[]  
BENIGN    TCP 57.144.140.141:443 (external) -> 103.52.142.15:42248 (internal)  
reasons=[]  
BENIGN    TCP 103.52.142.15:42248 (internal) -> 57.144.140.141:443 (external)  
reasons=[]  
BENIGN    TCP 155.133.225.21:443 (external) -> 103.52.142.15:34691 (internal)  
reasons=[]  
BENIGN    TCP 103.52.142.15:34691 (internal) -> 155.133.225.21:443 (external)  
reasons=[]  
BENIGN    TCP 103.52.142.15:52540 (internal) -> 8.222.147.122:443 (external) r  
easons=[]  
BENIGN    TCP 8.222.147.122:443 (external) -> 103.52.142.15:52540 (internal) r  
easons=[]  
BENIGN    TCP 103.52.142.15:56936 (internal) -> 57.144.140.145:443 (external)  
reasons=[]  
BENIGN    TCP 57.144.140.145:443 (external) -> 103.52.142.15:56936 (internal)  
reasons=[]  
BENIGN    TCP 57.144.140.145:443 (external) -> 103.52.142.15:56936 (internal)  
reasons=[]  
BENIGN    TCP 103.52.142.15:56936 (internal) -> 57.144.140.145:443 (external)  
reasons=[]  
[
```

Figure 4.5: Suricata Running Independently

The Figure 4.5 showcases Suricata running and processing traffic and Flagging packets as "Benign" or "Malicious".

4.4.4 Machine Learning Analysis Engine Overview

The Machine Learning Analysis Engine (MLAE) is the core component of the proposed framework, as it processes the predictions of machine learning models with alert logs from the signature-based IDS (e.g., Suricata) to generate confidence scores that quantify the likelihood of malicious nature of a packet flow, enabling adaptive decision-making.

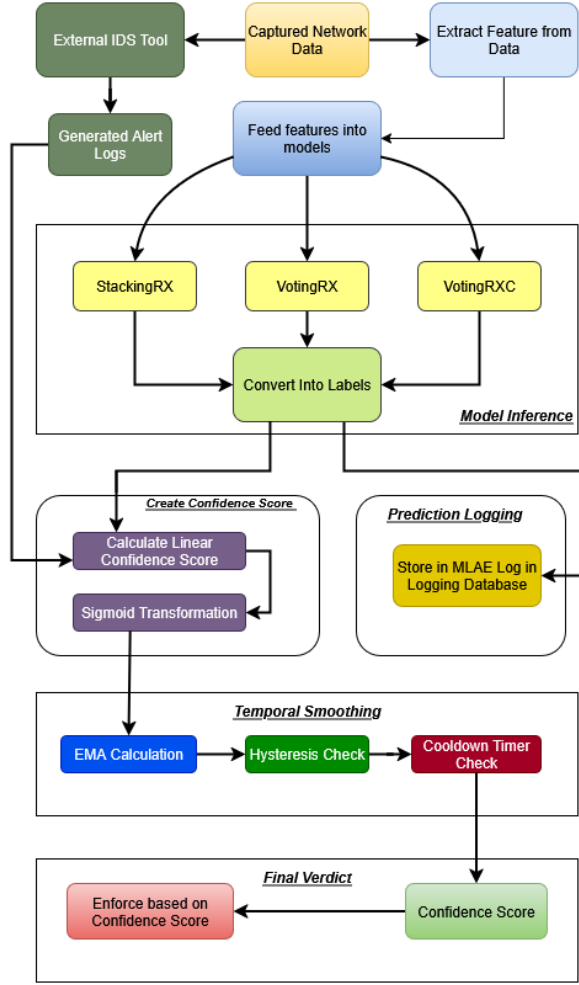


Figure 4.6: MLAE WORKFLOW

Figure 4.6 displays the working procedure of the MLAE. Preprocessing is done on captured network data to extract relevant features, which are then fed into three models: stackingRX, votingRX, and votingRXC that produce label predictions such as “Benign” or ”Malicious”. These outputs are combined with Suricata’s independent signature alerts and converted into a feature vector. A linear confidence score is calculated from this vector, where indicators such as Suricata malicious flags or consistent non-benign ML predictions increase the score. Then the raw score is normalized using a sigmoid transformation to produce a bounded confidence value between 0 and 1 to ensure precision in the decision. Then an exponential moving average (EMA) is applied, followed by hysteresis and cooldown checks(not flapping between the ALLOW/BLOCK status of the IP for 10 seconds) which filter out short-lived fluctuations and enforce temporal consistency in decisions. The final verdict ALLOW or BLOCK is derived from the smoothed confidence and the details(e.g., confidence score, model predictions, external IDS alert logs) of the decision are logged in the logging database for auditing later, and then based on the confidence score, actions are immediately applied to update firewall rules or connection states to reflect the decision.

```

{
  "confidence": {
    "ema": 0.9508011615638444,
    "mapping": {
      "confidence": 0.99127961562874,
      "features": {
        "RXC_non_benign": 1,
        "RX_non_benign": 1,
        "RXv_non_benign": 1,
        "suri_any_mal": 1,
        "suri_ratio": 0.6666666666666666
      },
      "linear_score": 4.733333333333333,
      "linear_terms": {
        "RXC_voting": 1,
        "RX_stacking": 0.8,
        "RX_voting": 0.6,
        "bias": 0,
        "suri_any_mal": 1.2,
        "suri_ratio": 1.1333333333333333
      }
    },
    "raw": 0.99127961562874
  },
  "mlae": {
    "RXC_voting": "Malicious",
    "RX_stacking": "Malicious",
    "RX_voting": "Malicious",
    "last_ts": "2025-10-07T02:29:54.646611+00:00"
  },
  "suricata": {
    "benign_count": 22,
    "mal_count": 44
  }
}

```

```

{
  "confidence": {
    "ema": 0.9168273035060777,
    "mapping": {
      "confidence": 0.9168273035060777,
      "features": {
        "RXC_non_benign": 1,
        "RX_non_benign": 1,
        "RXv_non_benign": 1,
        "suri_any_mal": 0,
        "suri_ratio": 0
      },
      "linear_score": 2.4,
      "linear_terms": {
        "RXC_voting": 1,
        "RX_stacking": 0.8,
        "RX_voting": 0.6,
        "bias": 0,
        "suri_any_mal": 0,
        "suri_ratio": 0
      }
    },
    "raw": 0.9168273035060777
  },
  "mlae": {
    "RXC_voting": "Malicious",
    "RX_stacking": "Malicious",
    "RX_voting": "Malicious",
    "last_ts": "2025-10-07T01:44:15.096643+00:00"
  },
  "suricata": {
    "benign_count": 0,
    "mal_count": 0
  }
}

```

Figure 4.7: Extended Details of Decisions

The Figure 4.7 showcases the details of two decisions. The confidence score, linear transformed values, the model predictions and Suricata alerts. The left decision of the figure shows both Suricata and models have flagged the connections traffic flow as malicious and the corresponding confidence score. On the right, it shows only the models have flagged the connections traffic flow as malicious where as Suricata flagged it as benign.

4.4.5 Runtime Processes

- The NetMan application written in Flask, provides the UI, packet capture threads using Scapy library, Admin panel, APIs, sqlite database for logging and functions for ARP discovery, reverse DNS lookup, segmentized access control, the models and the decision making loop that reads the Suricata logs CSV and MLAE predictions to compute decisions.
- The Suricata IDS runs with custom configuration and rule sets . It sniffs live packets and logs JSON events into eve.json. A flagger script then reads the Suricata eve.json, and correlates flows. Labeled packets (malicious or benign) are continuously written into a CSV file. Flushing occurs every few seconds for stability.

4.4.6 Sequence of Operations

- **Startup:** The `launch.sh` script initializes the system by starting Suricata and the NetMan Flask application. Suricata loads its rules, identifies the active interface, and launches the real-time flagger. NetMan configures its SQLite

database (WAL mode) and starts background workers including the DB writer, reverse DNS resolver, and ARP scanner.

- **Capture and Alerts:** Suricata inspects network traffic in real time in its own stream. Its flagger parses alerts from `eve.json`, builds directional flow keys (including reverse keys for NAT and asymmetric flows), and appends labeled rows (MALICIOUS or BENIGN) into `packet_flags.csv`. In parallel, NetMan spawns its own packet capture thread powered by Scapy with BPF filters that exclude noise (loopback, router IP, NetMan’s own web port, ARP, SSDP, mDNS). Parsed packets are queued into SQLite via a batch writer for efficiency.
- **Telemetry and Zero-Trust Tracking:** NetMan maintains live network state through continuous monitoring. ARP scanning discovers new peers, reverse DNS resolution maps external IPs to domain names, and trusted suffixes can auto-authorize known domains. Connection status (authorized, unknown, revoked) is tracked in the database, with expired peers removed after a 5-minute inactivity window. All packet activity is logged in the `netman.sqlite` db.
- **MLAE Predictions:** The Machine Learning Analysis Engine logs model outputs (`stackingRX`, `votingRX`, and `votingRXC`) into the `MLAELog` table. Input flows undergo pre-processing (normalization, cleaning, dimensionality reduction) before inference. Each record is timestamped and associated with the corresponding flow.

Oct 07, 2025, 08:35:09 AM	192.168.0.111	57.144.140.141	Benign	Benign	Benign
Oct 07, 2025, 08:35:09 AM	57.144.140.141	192.168.0.111	Benign	Benign	Benign
Oct 07, 2025, 08:35:09 AM	192.168.0.111	57.144.140.141	Benign	Benign	Benign
Oct 07, 2025, 08:35:08 AM	155.133.225.21	192.168.0.111	Malicious	Malicious	Malicious
Oct 07, 2025, 08:35:08 AM	192.168.0.111	155.133.225.21	Malicious	Malicious	Malicious
Oct 07, 2025, 08:35:07 AM	162.159.61.4	192.168.0.111	Benign	Malicious	Benign
Oct 07, 2025, 08:35:07 AM	192.168.0.111	162.159.61.4	Benign	Benign	Benign
Oct 07, 2025, 08:35:07 AM	155.133.225.21	192.168.0.111	Malicious	Malicious	Malicious
Oct 07, 2025, 08:35:07 AM	192.168.0.111	155.133.225.21	Malicious	Malicious	Malicious
Oct 07, 2025, 08:35:07 AM	162.159.61.4	192.168.0.111	Benign	Malicious	Benign
Oct 07, 2025, 08:35:07 AM	192.168.0.111	162.159.61.4	Benign	Benign	Benign
Oct 07, 2025, 08:35:07 AM	104.18.32.47	192.168.0.111	Benign	Benign	Benign

Figure 4.8: Model predictions on captured Packet IPs

The Figure 4.8 showcases model predictions of three models: `stackingRX`, `votingRX`, and `votingRXC` on captured traffic flows.

- **Fusion Decision and Pattern Validation:** A background loop executes every 5 seconds, aggregating Suricata alerts from `packet_flags.csv`, then fetching the most recent MLAE predictions per IP, and applies weighted voting for malicious verdicts block malicious packet source IP, while benign verdicts contribute equivalent points to allow. Each decision includes evidence and weights, is stored in the database for retrieval and is visualized in the UI.
- **Enforcement:** Decisions marked as BLOCK are dynamically applied at the firewall level through `nftables` rules, allowing the framework to automatically restrict network access for identified malicious hosts. The enforcement

operates within the OS kernel, ensuring persistence even after system restarts. Manual Overrides are provided to manually remove or reinstate blocking rules when necessary. The framework enforces only those block decisions whose confidence scores remain consistently above the enforcement threshold (confidence score greater than 0.90), as determined by the Exponential Moving Average (EMA) stability filter. All enforcement actions are logged for auditing and forensic analysis.

95.173.216.34	BLOCK	91.7%	✓	View	Unblock
47.79.65.75	BLOCK	91.1%	✓	View	Unblock
67.215.246.10	BLOCK	90.0%		View	Block
35.227.59.57	BLOCK	86.1%		View	Block
185.157.221.247	BLOCK	85.5%		View	Block

Figure 4.9: Enforcement by ML Analysis Engine

The Figure 4.9 shows the enforcement page of the prototype based on the decisions generated by the Machine Learning Analysis Engine (MLAE), where multiple IP addresses have been flagged as malicious with high confidence scores. However, not all “BLOCK” decisions are immediately enforced. This selective enforcement occurs because the MLAE applies an Exponential Moving Average (EMA) which continuously tracks the stability of the confidence score over time. So only when the averaged confidence exceeds the enforcement threshold is the decision marked as stable and enforced appropriately in the firewall. This way false positives are minimized by ensuring that short-lived anomalies do not trigger premature blocking.

4.4.7 Performance and Reliability Considerations

- Scapy’s BPF filters reduce noise and avoid interfering with NetMan’s own traffic.
- CSV file is written as append-only with periodic flushing to avoid data loss, ensuring crash safety and atomicity.
- Sqlite database in WAL mode enables concurrent read/write operations while maintaining ACID compliance and improving I/O efficiency.
- Database entries are entered as batches improve performance and consistency.

4.4.8 Prototype Limitations

- The models focus on packet and flow-level features. They do not yet account for encrypted traffic patterns, evasion techniques, or multi-stage attack campaigns.
- The MLAE models and fusion decision calibration were trained and tested on BCCC-CIC-IDS2017 only. This limits generalization to modern traffic patterns (e.g., encrypted traffic, IoT protocols) that are underrepresented in the dataset.

- The decision loop is designed for lab-scale throughput (tens/hundreds of Mbps). It has not yet been validated for high-speed enterprise networks or distributed deployments.
- Segmentation zones are policy-driven and are merely abstract representations. Data thresholds and port restrictions per segmentation are planned improvements.

4.4.9 Outcome

The prototype demonstrates the framework in action:

- The NetMan module monitors network state, manages telemetry, and enforces policy.
- The Suricata module provides independent signature based validation.
- The flagger turns Suricata's alerts into a continuous CSV stream.
- NetMan sends processed raw data to MLAE module for predictions of each model.
- The fusion decision loop combines ML predictions with Suricata evidence into auditable, explainable decisions.

This implementation shows how Zero-Trust controls, machine learning, and independent IDS can be integrated into a single coherent system for network monitoring and automated response.

4.5 Data Collection

For datasets, we chose well known cybersecurity datasets that come from credible sources, which are CIC-IDS2017 [11], BCCC-CIC-IDS2017 [43], The CAIDA Anonymized Internet Traces Dataset and UNSW-NB15 [4] [7] [12] [9] [20]. After thorough testing, the BCCC-CIC-IDS 2017 dataset was chosen as the primary source of data due to its comprehensive labeling, diverse attack types, and real-world network traffic representation. The BCCC-CIC-IDS 2017 dataset is a labeled, bi-directional flow-based dataset containing both benign and malicious instances of network activity. It includes modern 13 common attacks such as DoS (GoldenEye, Hulk, Slowhttptest, Slowloris), DDoS, PortScan, Botnet (ARES), Web Attacks (Brute Force, XSS, SQL Injection) and Brute Force. This makes it particularly suitable for training multi-class classification models that can distinguish between various types of cyber threats and normal traffic including flow-level statistical features. Also, the BCCC-CIC-IDS2017 dataset contains over 2.4 million labelled records, offering a large-scale and balanced environment for determining the performance of intrusion detection systems. Its high-quality labeling ensures minimal ambiguity in class definitions, which is essential for supervised learning tasks. Furthermore, the dataset mimics real-world traffic patterns and includes synthetic yet realistic attack behaviors, making it ideal for simulating practical security events. In contrast, the CAIDA dataset primarily consists of unlabeled network traffic traces collected from backbone internet infrastructure. While valuable for unsupervised learning or network behavior studies, the lack of labeled attack instances severely limits its applicability in training supervised learning which is not suited to our approach without further processing. Similarly the UNSW-NB15 dataset, despite being labeled, contains relatively fewer attack samples compared to BCCC-CIC-IDS2017. Moreover, some of the attack patterns in UNSW-NB15 are outdated and may not represent contemporary cyber threats effectively. With such considerations, the BCCC-CIC-IDS2017 dataset became the most suitable option for our research due to its attributes contributing greatly to test the proposed Avantgarde framework in detecting anomalies in networks with high accuracy and minimal false positives.

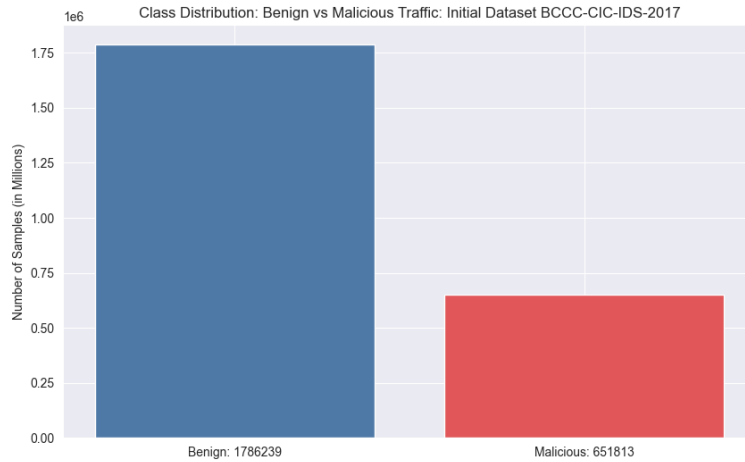


Figure 4.10: Initial Dataset Distribution

Here in the figure 4.10, represents the initial distribution of the BCCC-CIC-IDS2017 dataset before any data reduction and pre-processing was done.

4.5.1 Data Reduction & Pre-processing

In order to prepare the dataset for effective model training and testing, several data transformation techniques were applied. To start with, all the Benign and different malicious traffic data from BCCC-CIC-IDS-2017 dataset's CSV files were concatenated into one single Pandas DataFrame. After that, due to extremely high occurrence of the Benign class data, it was reduced to 100000 instances to enhance the balance of the dataset for ML model training. After that, the categorical variables such as network protocols and traffic labels were transformed into numerical representations using the `fit_transform()` function from `LabelEncoder()` in the `sklearn.preprocessing` library. Network protocols were encoded as 0 for TCP and 1 for UDP, while traffic labels were encoded to values ranging from 0 to 13, representing the 14 traffic label features present in the dataset. Next, we eliminated redundant features like, 'flow_id', 'timestamp', 'src_ip', 'dst_ip'. Still the dataset had 117 features left, many of which carried little information and many with high correlation between each other. So, to select the ideal feature set for our model training, we employed the Information Gain method, using the `mutual_info_classif()` function from `sklearn.feature_selection` library. After getting the information gain for each feature we chose the cutoff point of the information gain to be 0.8. Which left us with 32 features. We have also tried going with 0.5 and 0.7 information gain. But we got the best results in the test set and live traffic testing using these 32 features.

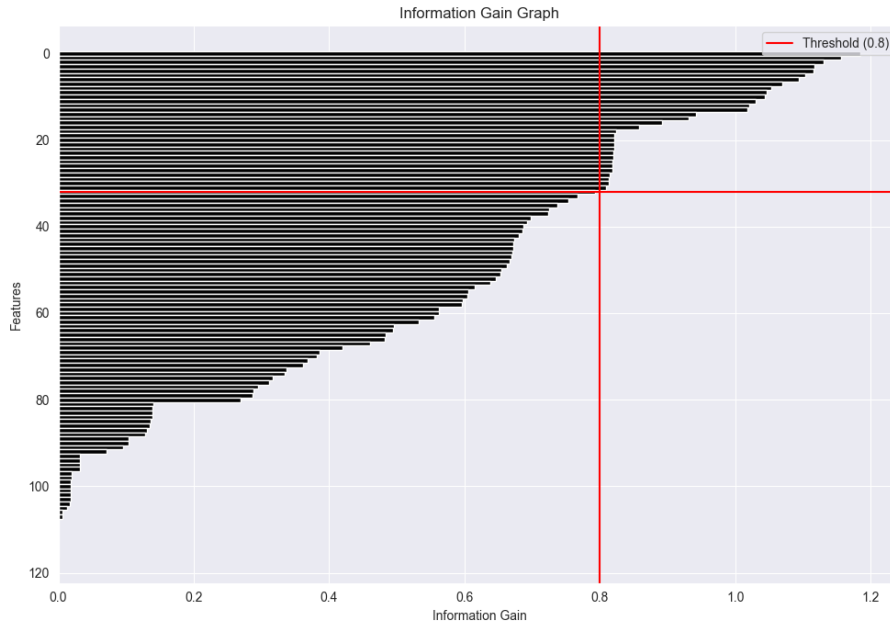


Figure 4.11: Feature Importance Graph

Here, the figure 4.11 depicts the threshold of 0.8 information gain with the vertical red line. And the selected features are the ones above the horizontal red line, which have the highest information gain scores.

Table 4.1: Selected Features and Their Information Gain

Selected Features	Information Gain
total_header_bytes	1.185222
mean_header_bytes	1.156638
fwd_total_header_bytes	1.131019
fwd_packets_IAT_max	1.116396
fwd_packets_IAT_total	1.115694
duration	1.102751
fwd_mean_header_bytes	1.093770
std_header_bytes	1.069071
fwd_packets_IAT_mean	1.052732
fwd_packets_rate	1.045990
packets_rate	1.043270
bwd_packets_rate	1.030247
bwd_total_header_bytes	1.020076
bwd_mean_header_bytes	1.018142
bwd_packets_IAT_max	0.941632
bwd_packets_IAT_total	0.930814
dst_port	0.892415
bwd_packets_IAT_mean	0.858020
packet_IAT_total	0.824525
payload_bytes_std	0.821687
fwd_payload_bytes_std	0.821318
payload_bytes_variance	0.820883
bwd_payload_bytes_std	0.820649
total_payload_bytes	0.820060
avg_segment_size	0.819246
payload_bytes_mean	0.818987
fwd_payload_bytes_mean	0.818633
bwd_payload_bytes_mean	0.818206
payload_bytes_max	0.814074
fwd_payload_bytes_max	0.813410
bwd_payload_bytes_max	0.813142
packet_IAT_max	0.809713

Here, in table 4.1, we have shown the selected features for our model training alongside their information gain scores.

Next, we split the dataset in 80:20 ratio into a test and train set. To improve on the imbalance of different classes in the training set, we employed the **SMOTE** method from `imblearn.over_sampling` for class balancing. Which synthetically increases the amount of instances of the classes which show up fewer times. Here, we used **SMOTE** to over-sample each class to at least 10000 instances.



Figure 4.12: Training set Class Distribution Before SMOTE

In the figure 4.12, we can see the train-set's class distribution of different attack types. The test-set shows a high imbalance because some of the attack types like **DoS Hulk**, and **DDoS LOIT** require to send high number of packets to overwhelm the victims network resources. Similarly, for Port Scan attacks send a huge number of small packets to generate short-lived connections. to discover open ports. Whereas, attacks like **Web XSS**, **Web Brute Force**, and **SSH Patator** are stealthy attacks which send fewer packets to avoid suspicion, which causes these classes to have small number of instances. However if we train our models on too little number of instances, it can cause confusion to in model's detection. So we synthetically over sampled the minority classes to at least 10000 instances using SMOTE to train the models to learn attack types effectively, which is depicted through the figure 4.13, showing the new class distribution after SMOTE.

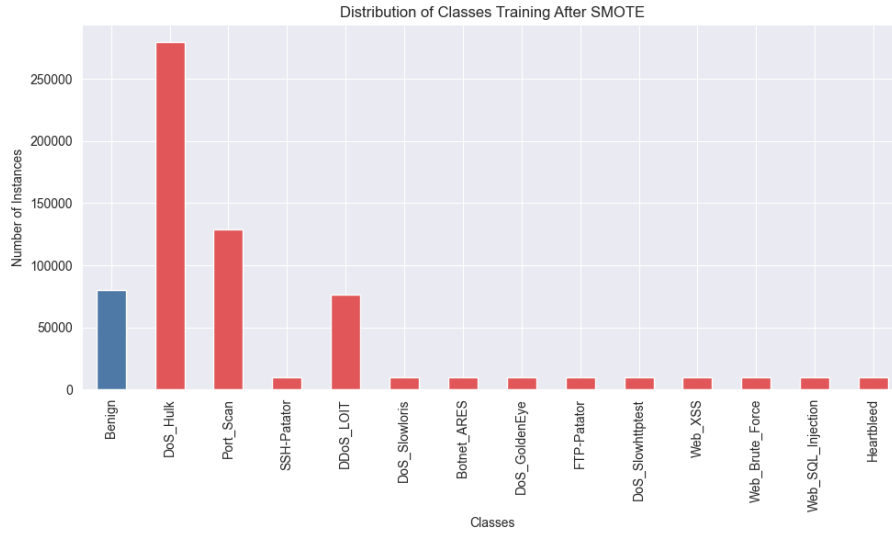


Figure 4.13: Training set Class Distribution After SMOTE

4.5.2 Summary of Preprocessed Data

- After pre-processing, the dataset was reduced to 32 features.
- Initial dimensions of the full dataset: $2,438,052 \times 122$.
- Final dimensions of the dataset: $751,813 \times 32$ (without target feature).
- The dataset was split into train and test sets using an 80:20 ratio stratified by labels columns.
- Train-set dimensions after SMOTE: $665,036 \times 32$.
- Test-set dimensions: $150,363 \times 32$.

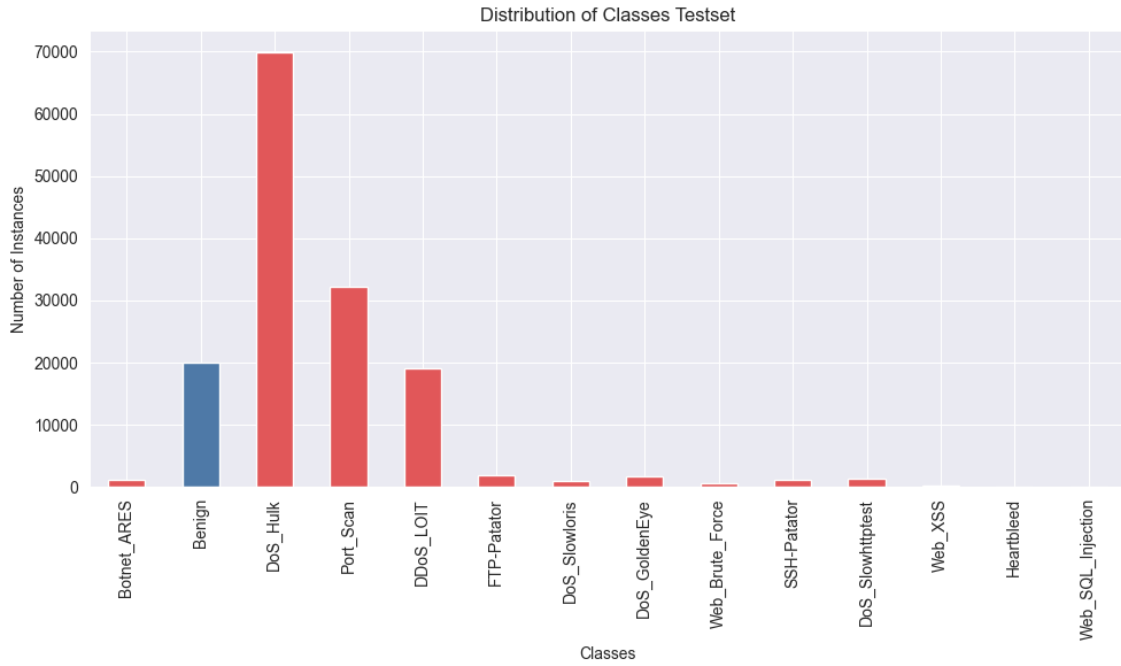


Figure 4.14: Test set distribution after 80:20 splitting

In the figure 4.14, we can see the test-set's class distribution of different attack types, which is imbalanced in the same way as the pre-SMOTE training-set. This is caused by the same reason explained earlier, which is the different requirements to carry out each attacks. Some attacks require high amount of packets to numb the receiver's resources, while some use stealthy attacks using few packets and focusing more on exploiting vulnerabilities.

4.6 Implementation of Models

Random Forest, XGBoost, and CatBoost are selected to train our models because of their demonstrated performance in handling high-dimensional, noisy, and imbalanced data like network traffic data. These algorithms are of the most effective members of the tree-based ensemble learning algorithms, which are well-suited for handling data complexity, heterogeneity, and high class imbalance, all of which are present in the BCCC-CIC-IDS2017 dataset. Especially compared to neural networks which require balanced, and homogeneous datasets to achieve high accuracy. Moreover Random Forest, XGBoost and CatBoost are explainable by design providing direct insights into decision boundaries, great for our use case. As the models trained with these algorithms also offer fast inference times and scalability, enabling real-time detection, crucial for the proposed Avantgarde framework. Now lets explore how these algorithms were used to develop our models:

Random Forest

Random Forest is an ensemble learning ML algorithm that builds multiple decision trees using different subsets of the training data and input features. It aggregates the predictions of these individual trees through majority voting for classification tasks. In the context of network anomaly detection, each tree is trained to differentiate between normal and anomalous traffic patterns. The final decision is made by combining the outputs of all trees, improving generalization and reducing overfitting [1].

Here the model was configured with 100 decision trees (`n_estimators=100`), a fixed random seed (`random_state=42`), and full parallelization (`n_jobs=-1`), the Random Forest features randomness to reduce overfitting on our high-dimensional tabular traffic data.

XGBoost

Extreme Gradient Boosting (XGBoost) is a scalable and efficient implementation of gradient boosting that sequentially builds decision trees to correct the errors of previous ones. It uses regularization to prevent overfitting and supports parallel processing and GPU acceleration for high-speed performance. In anomaly detection, XGBoost learns from the residuals of previous trees to refine the classification of network traffic, enhancing both precision and recall [6].

Here the mode was used in GPU-accelerated mode (`tree method='gpu hist'`, `predictor='gpu predictor'`), with its built-in label encoder disabled (`use label encoder=False`) and optimized using multi-class log-loss (`eval metric='mlogloss'`) to refine successive trees' residuals quickly.

CatBoost

CatBoost is a gradient boosting algorithm developed to handle categorical features efficiently without extensive pre-processing. It builds an ensemble of trees using ordered boosting, which reduces prediction shift and improves generalization. For

network intrusion detection, CatBoost automatically encodes categorical attributes in the traffic dataset and trains multiple trees to distinguish malicious flows from benign ones [10].

This model uses ordered boosting on the GPU (task type='GPU') for 500 iterations (iterations=500), with a learning rate of 0.05 and maximum tree depth of 6. Its native handling of categorical features eliminated additional preprocessing overhead.

Voting

We used the **Voting Classifier** from `sklearn.ensemble`, to combine all combinations of all the three base models (Random Forest, XGBoost, and CatBoost), to provide a more precise prediction. Here, we used soft voting, where each base model provides a class probability and the voting method takes the weighted average of the probabilities to classify the instance. The process is visually explained through the figure 4.15:

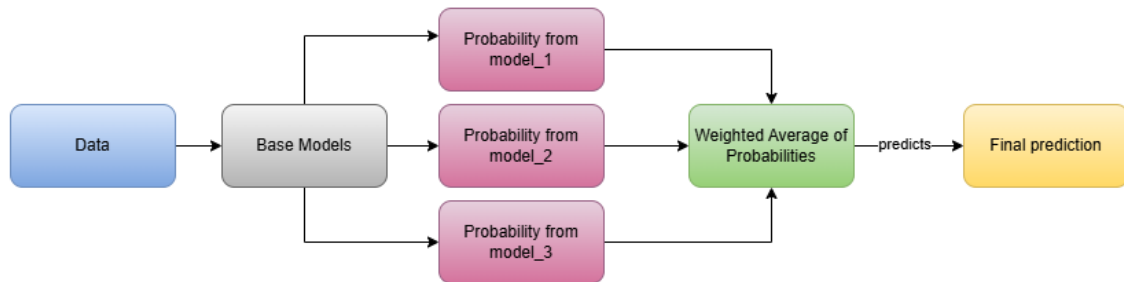


Figure 4.15: Voting Diagram

Stacking

Next, we used the **Stacking Classifier** from `sklearn.ensemble`, to again train all the combinations of the base models (Random Forest, XGBoost, and Cat Boost). Here in this method each of the models are trained in layers, in the base five times, one for each fold as we are using 5 fold cross validation. Then base models each generate a prediction, which is used as an input feature for the final estimator, which in this case is **Logistic Regression**. The final estimator, learns to best combine and select the results from the base models' predictions and produces the final prediction. The stacking process is explained visually through the diagram in figure 4.16 :

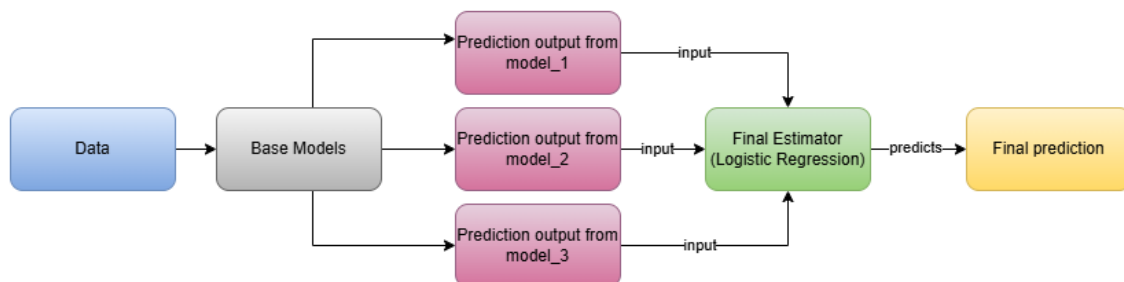


Figure 4.16: Stacking Diagram

Chapter 5

Result Analysis

5.1 Performance Evaluation of ML Models

This section discusses the performance of the ML models trained and chosen for the Avantgarde framework, based on performance metrics like accuracy, recall, precision, and f1-score.

Testbench

We trained and tested the models on a PC with an AMD Ryzen 5 3600 Processor which runs at 3.6 - 4.4 GHz, a NVIDIA RTX 3070 GPU with 8GB of VRAM and 32 GB of DDR4, 3200 MHz RAM.

Random Forest:

While training the Random Forest it took around 28 seconds for it to train on 665,036 packets. While testing on the test set it took around 1.5 seconds for the model to predict outcomes on 150,363 network packets.

Confusion Matrix: Random Forest Classifier

Benign	19906	1	4	6	41	7	5	2	1	4	4	14	0	5
Botnet_ARES	3	1099	0	0	0	0	0	0	0	0	0	0	0	0
DDoS_LOIT	0	0	19147	0	0	0	0	0	0	0	0	0	0	0
DoS_GoldenEye	0	0	0	1642	5	16	10	0	0	0	0	0	0	0
DoS_Hulk	42	0	0	8	69796	1	1	0	0	0	0	0	0	0
DoS_Slowhttptest	7	0	0	6	0	1337	22	0	0	0	0	0	0	0
DoS_Slowloris	5	0	0	2	0	28	1000	0	0	0	0	0	0	0
FTP-Patator	0	0	0	0	1	0	0	1905	0	0	0	0	0	0
Heartbleed	0	0	0	0	0	0	0	0	2	0	0	0	0	0
Port_Scan	3	0	0	0	0	0	0	0	0	32259	0	1	0	2
SSH-Patator	0	0	0	0	0	0	0	0	0	0	1190	0	0	0
Web_Brute_Force	4	0	0	0	1	1	0	0	0	0	0	437	0	104
Web_SQL_Injection	0	0	0	0	0	0	0	0	0	0	0	0	5	0
Web_XSS	2	0	0	0	0	0	0	0	0	0	0	51	0	218

Actual Class

Benign Botnet_ARES DDoS_LOIT DoS_GoldenEye DoS_Hulk DoS_Slowhttptest DoS_Slowloris FTP-Patator Heartbleed Port_Scan SSH-Patator Web_Brute_Force Web_SQL_Injection Web_XSS

Predicted Class

Figure 5.1: Confusion matrix of Random Forest Classifier

In the figure 5.1, we can see the Random Forest model's confusion matrix. From that, we can see the model can differentiate between Benign and different malicious classes with quite high precision. However, while detecting attack types with comparatively less number of instances the model gets confused. For example, even though the model successfully detected most **Web_XSS**, and **Web_Brute_Force** attacks as threats, it misclassified a lot of **Web_XSS** attacks as **Web_Brute_Force** and vice-versa. Which affects the model's overall metrics as displayed in the table 5.1.

Table 5.1: Performance metrics for RandomForest model

Model	Accuracy	Macro Precision	Macro Recall	Macro F1-Score
Random Forest	99.72%	93.57%	96.55%	94.76%

However, we can understand how proficiently the model can detect Malicious traffic from Benign traffic from the figure 5.2, which shows the binary classification confusion matrix. Where it shows that the model misclassified only 66 attack packets out of 130363, while only 94 benign packets were misclassified out of 20000.

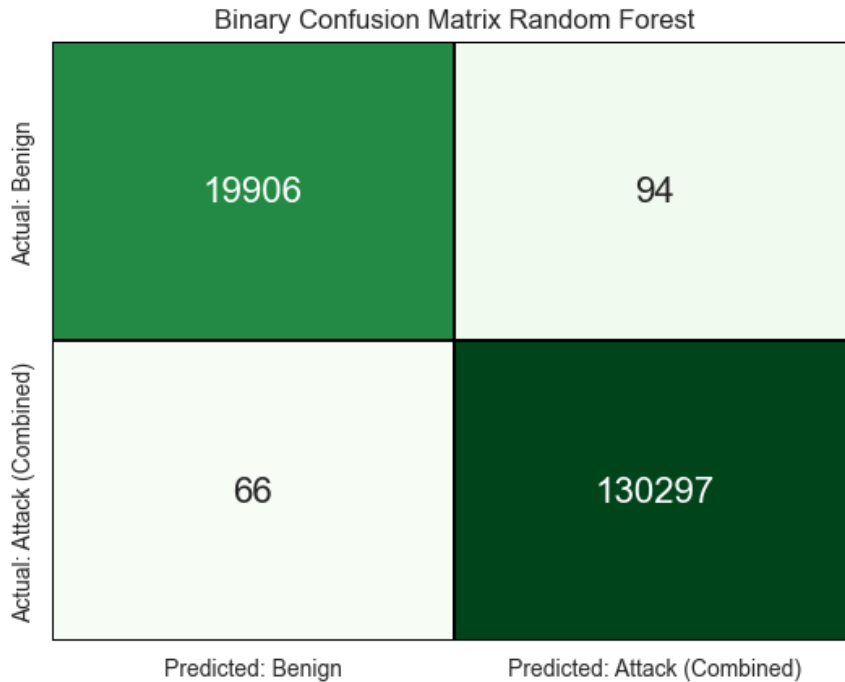


Figure 5.2: Confusion matrix of Random Forest Classifier (Binary Classification)

XGBoost:

The training time for XGBoost was 10 seconds, and testing on the same test-set as the previous model, it only took 356 ms.

Confusion Matrix: XGBoost Classifier

Actual Class	Benign	19787	1	1	3	53	12	29	1	1	6	2	41	21	42
	Botnet_ARES	0	1102	0	0	0	0	0	0	0	0	0	0	0	0
	DDoS_LOIT	12	0	19135	0	0	0	0	0	0	0	0	0	0	0
	DoS_GoldenEye	0	0	0	1628	4	27	14	0	0	0	0	0	0	0
	DoS_Hulk	32	0	0	4	69793	9	7	0	0	0	0	3	0	0
	DoS_Slowhttptest	5	0	0	5	2	1292	68	0	0	0	0	0	0	0
	DoS_Slowloris	5	0	0	3	1	30	995	0	0	0	0	1	0	0
	FTP-Patator	0	0	0	0	0	0	0	1906	0	0	0	0	0	0
	Heartbleed	0	0	0	0	0	0	0	0	2	0	0	0	0	0
	Port_Scan	2	0	0	0	0	0	0	0	0	32260	0	0	0	3
	SSH-Patator	0	0	0	0	0	0	0	0	0	0	1190	0	0	0
	Web_Brute_Force	4	0	0	0	0	0	1	0	0	0	0	412	0	130
	Web_SQL_Injection	0	0	0	0	0	0	0	0	0	0	0	0	5	0
	Web_XSS	2	0	0	0	0	0	0	0	0	0	0	56	0	213
		Benign	Botnet_ARES	DDoS_LOIT	DoS_GoldenEye	DoS_Hulk	DoS_Slowhttptest	DoS_Slowloris	FTP-Patator	Heartbleed	Port_Scan	SSH-Patator	Web_Brute_Force	Web_SQL_Injection	Web_XSS
		Predicted Class													

Figure 5.3: Confusion Matrix of XGBoost.

From the figure 5.3, we get the confusion matrix for the XGBoost model. In this case as well, we can see a similar pattern as the Random Forest models performance. However, the XGBoost model shows an increase in the false alarm rate as seen from the confusion matrix. And it significantly brings down the macro-precision and f1-score of the model, as illustrated in the table 5.2.

Table 5.2: Performance metrics for XGBoost model

Model	Accuracy	Macro Precision	Macro Recall	Macro F1-Score
XGBoost	99.57%	85.93%	95.74%	88.48%

However, the figure 5.4 displays the binary classifying confusion matrix of the XGBoost model. From here we can see, that the XGBoost classifier detects threats with higher accuracy than the random forest classifier, misclassifying only 62 attacks out of 130353. But the model shows higher vulnerability in terms of false alarms. As

the model misclassified 213 benign instances as attacks, which is significantly higher than random forest.

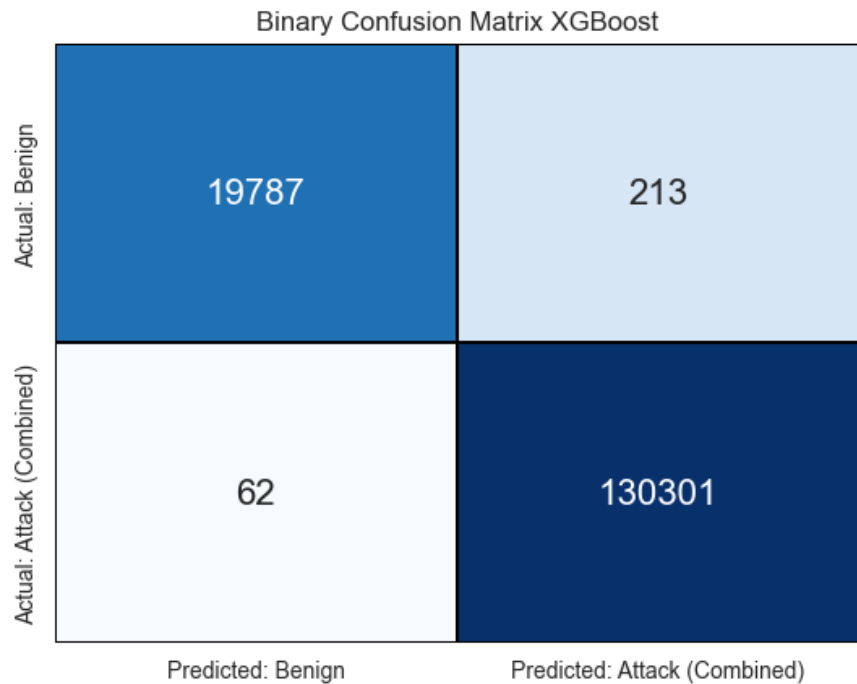


Figure 5.4: Confusion matrix of XGBoost Classifier (Binary Classification)

Cat Boost:

The model training took around 25 seconds while training was around 567ms.

Confusion Matrix: Cat Boost Classifier

Benign	19646	3	1	16	55	30	2	51	4	9	25	69	14	75
Botnet_ARES	0	1102	0	0	0	0	0	0	0	0	0	0	0	0
DDoS_LOIT	15	0	19132	0	0	0	0	0	0	0	0	0	0	0
DoS_GoldenEye	0	0	0	1610	14	28	8	0	0	0	0	0	13	0
DoS_Hulk	35	0	0	16	69781	13	0	0	0	0	0	0	0	3
DoS_Slowhttptest	14	0	0	8	38	1303	8	0	0	0	0	0	1	0
DoS_Slowloris	41	0	0	8	0	132	854	0	0	0	0	0	0	0
FTP-Patator	4	0	0	0	0	0	0	1902	0	0	0	0	0	0
Heartbleed	0	0	0	0	0	0	0	0	2	0	0	0	0	0
Port_Scan	8	0	0	1	9	0	0	0	0	32244	0	0	0	3
SSH-Patator	1	0	0	0	0	0	0	0	0	0	1189	0	0	0
Web_Brute_Force	2	0	0	0	2	0	0	0	0	0	0	392	1	150
Web_SQL_Injection	0	0	0	0	0	0	0	0	0	0	0	0	5	0
Web_XSS	1	0	0	0	0	0	0	0	0	0	0	21	1	248

Predicted Class

Figure 5.5: Confusion Matrix of CatBoost

The figure 5.5 illustrates the confusion matrix of the CatBoost classifier model. This model too continues to struggle to differentiate between **Web_XSS**, and **Web_Brute_Force** attack types, at the same time this model also struggles to differentiate between the different types of DoS attacks. Which makes the overall performance metrics of this model even worse compared to the XGBoost model as displayed in the table 5.3.

Table 5.3: Performance metrics for CatBoost model

Model	Accuracy	Macro Precision	Macro Recall	Macro F1-Score
CatBoost	99.37%	82.60%	95.33%	85.00%

Even though the CatBoost classifier's macro metrics in table 5.3 and confusion matrix in figure 5.5 give the idea that the model performs quite poorly compared to random forest and XGBoost classifiers. However, the binary classifying confusion matrix from figure 5.6 shows that this model too successfully detects threats 130242 out of 130363 instances. Which is a satisfactory performance.

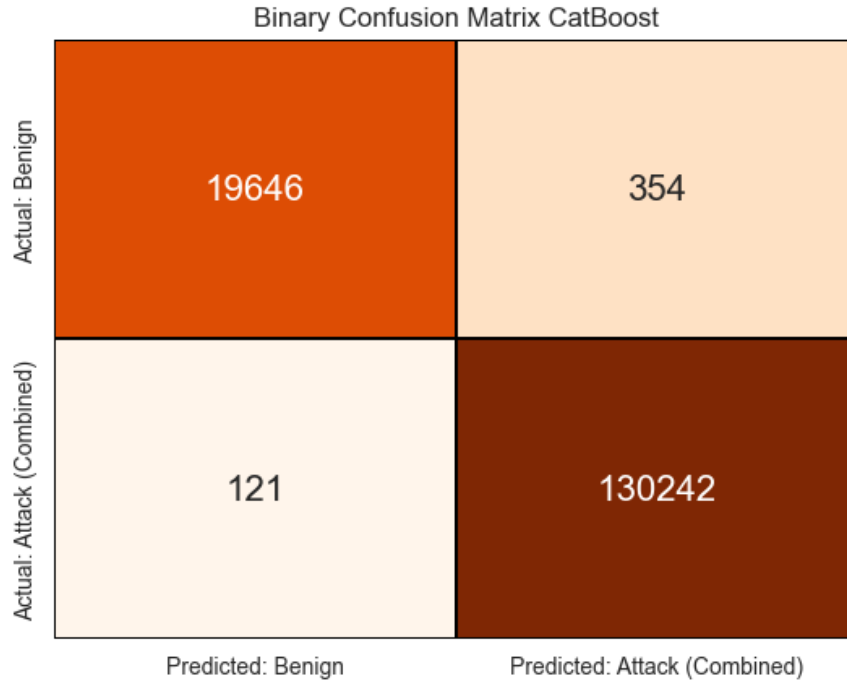


Figure 5.6: Confusion matrix of CatBoost Classifier (Binary Classification)

Combinations:

Next, we trained and tested all the combination of the previous three models, using both voting and stacking methods. Each of which have shown different results varying predictions compared to the standalone models in terms of better and worse, as the table 5.4 shows below:

Table 5.4: Macro Performance metrics of all trained Models

Model	Accuracy	Precision	Recall	F1-Score
StackingRXC	99.73%	92.44%	96.75%	94.22%
StackingRX	99.73%	92.44%	96.75%	94.22%
VotingRX	99.72%	91.50%	96.74%	93.62%
VotingRXC	99.70%	91.85%	96.68%	93.85%
VotingRC	99.71%	89.43%	96.65%	92.00%
StackingRC	99.73%	90.90%	96.56%	93.13%
Random Forest	99.72%	93.57%	96.55%	94.76%
StackingXC	99.61%	90.32%	96.35%	92.65%
VotingXC	99.57%	88.94%	96.27%	91.59%
XGBoost	99.57%	85.93%	95.74%	88.48%
CatBoost	99.37%	82.60%	95.33%	85.00%

In the table 5.4, we have the models sorted by the macro-recall and macro-precision values. As these two metrics show how proficient the models are in detecting real threats and avoiding false alarms. From all the combinations, we then finally chose the **StackingRXC**, **StackingRX**, and **VotingRX** models for our framework as these are the best performing models available.

Table 5.5: Binary Performance metrics of all trained Models

Model	Accuracy	Precision	Recall	F1-Score
StackingRXC	99.90%	99.93%	99.96%	99.94%
StackingRX	99.90%	99.93%	99.96%	99.94%
VotingRX	99.90%	99.92%	99.96%	99.94%
VotingRXC	99.88%	99.90%	99.96%	99.93%
VotingRC	99.88%	99.91%	99.95%	99.93%
StackingRC	99.90%	99.93%	99.95%	99.94%
Random Forest	99.89%	99.93%	99.95%	99.94%
StackingXC	99.84%	99.87%	99.95%	99.91%
VotingXC	99.82%	99.84%	99.95%	99.90%
XGBoost	99.82%	99.84%	99.95%	99.89%
CatBoost	99.68%	99.73%	99.91%	99.82%

Even though, we have chosen the best models based on their macro-metrics, we have also combined the performance metrics from the binary classification confusion matrices in the table 5.5. This table gives a better idea by communicating the proficiency of the models' capability of detecting generalized threats from normal network traffic.

Stacking [RFC, XGBC, CBC]:

The stacking method of combining the models took a little long, combining all three base models took around 4 minutes and 5 seconds to train. However, the testing time was only 5 seconds.

Confusion Matrix: Combined Stacking [RFC, XGBC, CBC]

Actual Class	Benign	19905	1	3	6	39	7	3	0	1	6	3	12	1	13
	Botnet_ARES	0	1102	0	0	0	0	0	0	0	0	0	0	0	0
	DDoS_LOIT	0	0	19147	0	0	0	0	0	0	0	0	0	0	0
	DoS_GoldenEye	0	0	0	1641	6	16	10	0	0	0	0	0	0	0
	DoS_Hulk	31	0	0	9	69803	1	1	0	0	0	0	0	0	3
	DoS_Slowhttptest	5	0	0	7	0	1336	24	0	0	0	0	0	0	0
	DoS_Slowloris	6	0	0	5	0	27	997	0	0	0	0	0	0	0
	FTP-Patator	0	0	0	0	0	0	0	1906	0	0	0	0	0	0
	Heartbleed	0	0	0	0	0	0	0	0	2	0	0	0	0	0
	Port_Scan	2	0	0	0	0	0	0	0	0	32260	0	1	0	2
	SSH-Patator	0	0	0	0	0	0	0	0	0	0	1190	0	0	0
	Web_Brute_Force	5	0	0	0	0	1	0	0	0	0	0	441	0	100
	Web_SQL_Injection	0	0	0	0	0	0	0	0	0	0	0	0	5	0
	Web_XSS	2	0	0	0	0	0	0	0	0	0	0	45	0	224
		Benign	Botnet_ARES	DDoS_LOIT	DoS_GoldenEye	DoS_Hulk	DoS_Slowhttptest	DoS_Slowloris	FTP-Patator	Heartbleed	Port_Scan	SSH-Patator	Web_Brute_Force	Web_SQL_Injection	Web_XSS
		Predicted Class													

Figure 5.7: Confusion Matrix StackingRXC [RFC, XGBC, CBC]

In the figure 5.7 we can find the confusion matrix of the StackingRXC model. Where we can see that combining all of the three models together using stacking method improves the model's detection of benign and malicious classes, alongside minimizing the issue of false alarms. Which is also displayed in the table 5.6 by improved macro-recall and precision values.

Table 5.6: Performance Metrics of StackingRXC [RFC, XGBC, CBC]

Model	Accuracy	Macro Precision	Macro Recall	Macro F1-Score
StackingRXC	99.73%	92.44%	96.75%	94.22%

From the confusion matrix in figure 5.7, it is tough to notice the performance difference from previous models. However, the binary classifying confusion matrix in figure 5.8 shows the performance improvement more clearly. As we can see only 51 misclassified attacks out of 130363 instances and only 95 false alarm out of 20000 benign packets.

Binary Confusion Matrix StackingRXC	
Actual:	Benign
	Attack (Combined)
Actual: Attack (Combined)	Benign
	Attack (Combined)

Figure 5.8: Confusion matrix of StackingRXC (Binary Classification)

Stacking [RFC, XGBC]:

Stacking Random Forest and XGBoost took around 3 minutes to train and took only 3 seconds to test.

Confusion Matrix: Combined Stacking [RFC, XGBC]	
Actual Class	Benign
	Botnet_ARES
	DDoS_LOIT
	DoS_GoldenEye
	DoS_Hulk
	DoS_Slowhttptest
	DoS_Slowloris
	FTP-Patator
	Heartbleed
	Port_Scan
	SSH-Patator
	Web_Brute_Force
	Web_SQL_Injection
	Web_XSS
	Benign
	Botnet_ARES
	DDoS_LOIT
	DoS_GoldenEye
	DoS_Hulk
	DoS_Slowhttptest
	DoS_Slowloris
	FTP-Patator
	Heartbleed
	Port_Scan
	SSH-Patator
	Web_Brute_Force
	Web_SQL_Injection
	Web_XSS

Figure 5.9: Confusion Matrix StackingRX [RFC, XGBC]

In the figure 5.9, we can see the confusion matrix of the StackingRX model, which combines the Random Forest and XGBoost model with the stacking method. The StackingRX model also shows very similar performance to the StackingRXC model which is the best performing model of all the models we have trained and tested. The similarity in performance can also be seen in the table 5.7.

Table 5.7: Performance Metrics of StackingRX [RFC, XGBC]

Model	Accuracy	Macro Precision	Macro Recall	Macro F1-Score
StackingRX	99.73%	92.44%	96.75%	94.22%

Here, in the figure 5.10 we see more easily that the StackingRX model performs exactly the same the StackingRXC model. Both show exactly 51 number of misclassifications while detecting threats and raised exactly 95 false alarms.

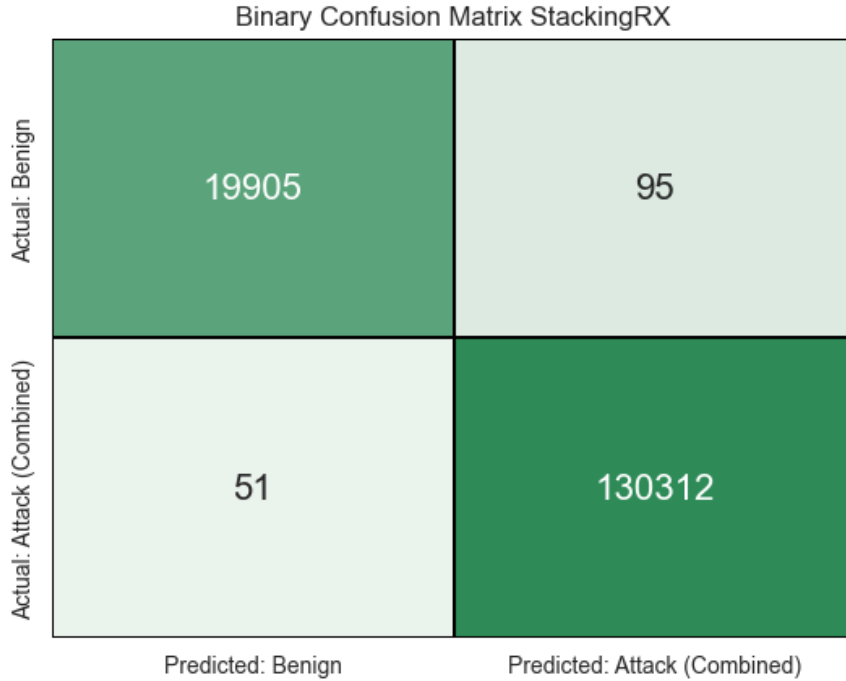


Figure 5.10: Confusion matrix of StackingRX (Binary Classification)

Voting [RFC, XGBC]:

This combination took around 54 seconds and testing took around 4 seconds.

Actual Class	Benign	Botnet_ARES	DDoS_LOIT	DoS_GoldenEye	DoS_Hulk	DoS_Slowhttptest	DoS_Slowloris	FTP-Patator	Heartbleed	Port_Scan	SSH-Patator	Web_Brute_Force	Web_SQL_Injection	Web_XSS
Benign	19894	1	3	3	42	7	6	0	1	6	1	16	2	18
Botnet_ARES	0	1102	0	0	0	0	0	0	0	0	0	0	0	0
DDoS_LOIT	0	0	19147	0	0	0	0	0	0	0	0	0	0	0
DoS_GoldenEye	0	0	0	1637	7	20	9	0	0	0	0	0	0	0
DoS_Hulk	32	0	0	8	69806	1	1	0	0	0	0	0	0	0
DoS_Slowhttptest	4	0	0	7	0	1337	24	0	0	0	0	0	0	0
DoS_Slowloris	6	0	0	3	1	26	999	0	0	0	0	0	0	0
FTP-Patator	0	0	0	0	0	0	0	1906	0	0	0	0	0	0
Heartbleed	0	0	0	0	0	0	0	0	2	0	0	0	0	0
Port_Scan	2	0	0	0	0	0	0	0	0	32260	0	1	0	2
SSH-Patator	0	0	0	0	0	0	0	0	0	0	1190	0	0	0
Web_Brute_Force	5	0	0	0	0	0	1	0	0	0	0	440	0	101
Web_SQL_Injection	0	0	0	0	0	0	0	0	0	0	0	0	5	0
Web_XSS	2	0	0	0	0	0	0	0	0	0	0	45	0	224
Predicted Class	Benign	Botnet_ARES	DDoS_LOIT	DoS_GoldenEye	DoS_Hulk	DoS_Slowhttptest	DoS_Slowloris	FTP-Patator	Heartbleed	Port_Scan	SSH-Patator	Web_Brute_Force	Web_SQL_Injection	Web_XSS

Figure 5.11: Confusion Matrix VotingRX [RFC, XGBC]

In the figure 5.11, we get the confusion matrix for the VotingRX. Which is our third best model according to macro-recall and precision metric. However these model too shows the weakness of misclassifying between different attack classes. The overall performance metrics of the model is displayed in the table 5.8.

Table 5.8: Performance Metrics of VotingRX [RFC, XGBC]

Model	Accuracy	Macro Precision	Macro Recall	Macro F1-Score
VotingRX	99.72%	91.50%	96.74%	93.62%

The figure 5.12 illustrates the confusion matrix in binary classification form, to give a better idea for the model's performance in detecting generalized threats from network traffic.

Binary Confusion Matrix VotingRX		
Actual: Benign	19894	106
Actual: Attack (Combined)	51	130312
	Predicted: Benign	Predicted: Attack (Combined)

Figure 5.12: Confusion matrix of VotingRX (Binary Classification)

Choosing the 4th Best Model:

From the initially chosen three models, we can see that the StackingRX and StackingRXC show the exact same behavior in detecting threats. Which is also confirmed by the confusion matrices of each model in figure 5.7 & 5.9. Thus, we have gone onto choose the 4th best performing model VotingRXC as our third model for the framework, dropping the StackingRXC model. Because even though both of the models had the exact same performance, the StackingRX model took less time for training and testing.

Voting [RFC, XGBC, CBC]:

The training of this combination took around 54 seconds and testing took around 4 seconds.

Confusion Matrix: Combined Voting [RFC, XGBC, CBC]

Benign	19865	1	3	6	38	6	10	1	1	4	3	23	1	38
Botnet_ARES	0	1102	0	0	0	0	0	0	0	0	0	0	0	0
DDoS_LOIT	0	0	19147	0	0	0	0	0	0	0	0	0	0	0
DoS_GoldenEye	0	0	0	1636	6	20	11	0	0	0	0	0	0	0
DoS_Hulk	31	0	0	10	69802	1	1	0	0	0	0	3	0	0
DoS_Slowhttptest	5	0	0	3	0	1341	23	0	0	0	0	0	0	0
DoS_Slowloris	6	0	0	2	0	29	998	0	0	0	0	0	0	0
FTP-Patator	0	0	0	0	0	0	0	1906	0	0	0	0	0	0
Heartbleed	0	0	0	0	0	0	0	0	2	0	0	0	0	0
Port_Scan	4	0	0	0	0	0	0	0	0	32258	0	0	0	3
SSH-Patator	0	0	0	0	0	0	0	0	0	0	1190	0	0	0
Web_Brute_Force	4	0	0	0	0	0	1	0	0	0	0	438	0	104
Web_SQL_Injection	0	0	0	0	0	0	0	0	0	0	0	0	5	0
Web_XSS	1	0	0	0	0	0	0	0	0	0	0	47	0	223

Predicted Class

Figure 5.13: Confusion Matrix VotingRXC [RFC, XGBC, CBC]

From the figure 5.13, we can see that like all the other models this too shows the weakness of missclassifying between different attack types. Which affects the macro-metrics as illustrated in the table 5.9.

Table 5.9: Performance Metrics of VotingRXC [RFC, XGBC, CBC]

Model	Accuracy	Macro Precision	Macro Recall	Macro F1-Score
VotingRXC	99.70%	91.85%	96.68%	93.85%

However, taking a look at the binary classification confusion matrix from figure 5.14, illustrates that this model too performs quite well, missing only 51 threats out of 130363 instances, at the same time missclassifying only 135 out of 20000 benign instances.

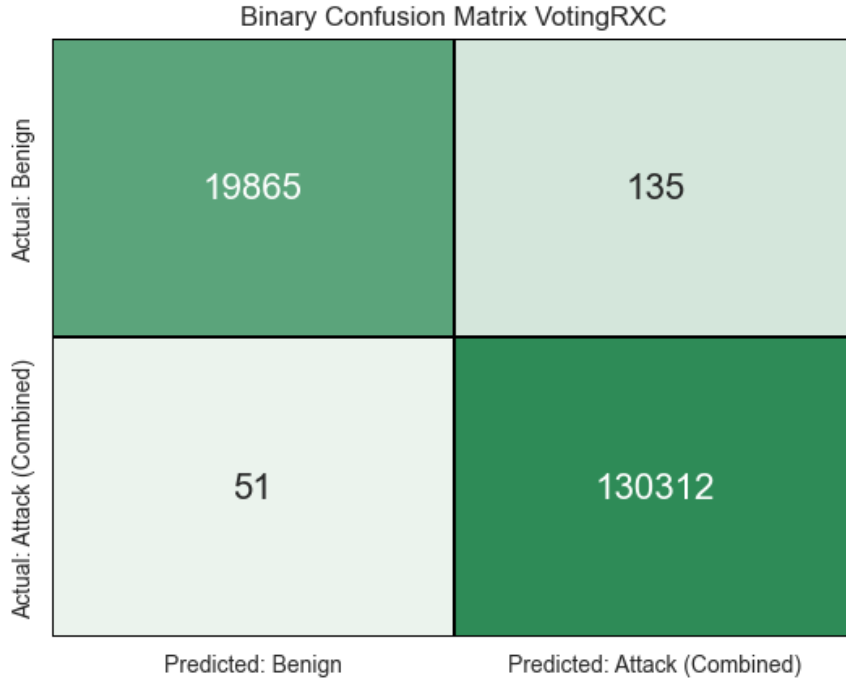


Figure 5.14: Confusion matrix of VotingRXC (Binary Classification)

Evaluation Summary

Finally, after evaluating all the models and adjusting for model variety, we have chosen 3 models for using in the Avantgarde framework. The models are StackingRX, VotingRX, and VotingRXC. All of which show excellent performance in all the metrics. All of which summarized down in the table 5.10:

Table 5.10: Performance Metrics of the Chosen Models

Model	Macro Accuracy	Macro Precision	Macro Recall	Macro F1-Score	Binary Accuracy	Binary Precision	Binary Recall	Binary F1-Score
StackingRX	99.73%	92.44%	96.75%	94.22%	99.90%	99.93%	99.96%	99.94%
VotingRX	99.72%	91.50%	96.74%	93.62%	99.90%	99.92%	99.96%	99.94%
VotingRXC	99.70%	91.85%	96.68%	93.85%	99.88%	99.90%	99.96%	99.93%

5.2 Analysis of Final Design

5.2.1 Description of the Final Design

The final design of Avantgarde is a hybrid, host-based intrusion detection & prevention system (HIDPS) that fuses signature-based detection of Suricata with predictions from multiple models. Key design elements include:

- **Machine Learning:** By training and testing various models on our dataset, the three best performing ML models: voting_RXC, stacking_RX, voting_RX were chosen to observe flow-level features in captured traffic and output categorical labels: benign or specific attack classes.
- **Suricata Integration:** Suricata runs independently, generating alert logs and verdicts. Its outputs are parsed and aggregated per external IP traffic flow as “malicious” or “benign” counts.
- **Machine Learning Analysis Engine (MLAE):** For each IP’s traffic flow, we build features from Suricata’s malicious/benign counts plus binary flags (non-benign flags) generated by ML predictions. These features are used to calculate confidence score using simple linear transformation and sigmoid mapping.
- **Temporal Stability:** The system maintains an Exponential Moving Average (EMA) of confidence score per traffic flow, and uses hysteresis and cool-down logic to decide when to flip between ALLOW or BLOCK to avoid oscillating or flapping.
- **Enforcement:** When a BLOCK decision is deemed credible (EMA passes enforcement threshold), the system writes a kernel level firewall rule so the hosts firewall blocks the traffic flows IP. Manual override via UI (block/allow) was implemented to supersede malinformed automated behavior.

5.2.2 Final Design Adjustments

During the course of the prototype development, several design adjustments were made to evolve the framework from its initial concept into a more robust and adaptive system. The key adjustments are summarized below:

1. From Static Weights to Fusion-Based Decisions

Initially, we considered a static weight assignment scheme where outputs of Suricata and the ML models would be linearly combined with pre-set weights. This was later replaced with a fusion decision process that computes a linear score from multiple features (e.g., Suricata mal/benign ratio, ML model outputs), then applying a sigmoid transformation to derive a confidence value, and then smooths it over time using an Exponential Moving Average (EMA). The EMA, combined with hysteresis and cooldown, prevents flapping or oscillating in decision-making and stabilizes block/allow enforcement.

2. Segmentation Adjustments

In the early stage, segmentation was binary, connections were marked either

as "blocked" or "allowed". The design was later expanded to include segmented connection states: unknown, authorized, and revoked. Unknown connections were permitted under restrictions (e.g., ports, payload limits, or protocol types), while authorized hosts enjoyed relaxed restrictions, and revoked hosts were fully blocked.

3. Cross-Checking with Multiple ML Models

Instead of depending on the sole prediction of one model, three different ML models (stackingRX, votingRX, votingRXC) were integrated into the MLAE. Their outputs were reduced into binary non-benign/benign indicators, which were then used as independent features in the fusion stage. This redundancy reduced the risk of a single model's bias or misclassification dominating the system's behavior.

4. Manual Overrides and Persistence

Manual block/unblock functionality was added to allow the administrator to override automatic decisions. These overrides were made persistent by synchronizing with the database and firewall layer, ensuring that user-defined choices could not be overwritten by automated enforcement.

5. Firewall Integration Enhancements

At first, enforcement relied solely on IP blocking. This was later extended with segmentation-aware firewall sets (unknown, authorized, revoked) using nftables on Linux. In Windows, existing TCP/UDP sessions were proactively torn down (via conntrack or Close-NetTCPConnection) to make enforcement immediate.

6. Logging and Explainability

MLAE predictions were only printed on the terminal. then for auditing and explainability, MLAE predictions were persistently logged, alongside Suricata verdicts and fusion decisions, allowing cross-analysis and audit. The fusion decision module also records per-feature attributions (linear score contributions, EMA, final decision), providing a minimal but interpretable explanation layer.

These adjustments transformed the design from a simple rule-plus-model approach into an adaptive HIDPS prototype reflecting Avantgarde, capable of balancing detection, stability, administrator control, and explainability.

5.2.3 Final Design Evaluation

In analyzing the Avantgarde framework’s final design, the following evaluation criteria were considered:

- **Accuracy:** Ability of the system to correctly classify and enforce ALLOW/BLOCK decisions with minimal false positives and false negatives.
- **Performance:** Computational overhead introduced by traffic capture, ML inference, external IDS tool logs parsing, decision loop, and database operations.
- **Feasibility:** Ease of integration into existing systems, maintainability, resilience against restart, and deployability on host hardware.
- **Cost:** Resource utilization (CPU, memory), development and maintenance complexity, and dependency requirements.

The implemented design of Avantgarde was analyzed against the above evaluation criteria. The summary is shown in table 5.11.

Table 5.11: Evaluation of Avantgarde Design Against Criteria

Criterion	Analysis
Accuracy	The hybrid design improves detection by combining Suricata’s high precision on known signatures with ML models that capture anomalous traffic. Temporal smoothing (EMA) reduces transient false positives. However, the current linear fusion method may underperform against highly sophisticated attack patterns.
Performance	Decision computations are lightweight, and ML inference is only applied on flow-level data, reducing per-packet overhead. Database batching improves I/O efficiency.
Feasibility	The system can run on general servers, workstations or normal pcs without requiring specialized hardware. Modularity of design allows for replacing models or changing the external IDS tool.
Cost	Development and debugging complexity is higher compared to signature-only systems. However, being based on open-source components (Suricata, Flask, SQLite, ML libraries) keeps licensing costs minimal. Resource usage remains moderate relative to more heavyweight IDS stacks.

5.2.4 Avantgarde Compared to Other Systems

Then table 5.12 displays the comparison of the final design of Avantgarde against several existing systems. References are drawn from known systems such as **OSSEC**, **Suricata**, **Snort**, **Wazuh**, and **Zeek**.

Table 5.12: Evaluation of Avantgarde Design Against Other Systems

System	Architecture	Data Dependency	Detection / Prevention
OSSEC [41]	Host-based and agent-driven. Configured via XML rules and decoders.	Uses host logs, system events, and file integrity checks. Limited network data usage.	Performs log-based intrusion detection and file integrity monitoring. Prevention possible via active custom scripts.
Suricata [40]	Network-based. Supports multithreading and inline packet capture with deep flow inspection.	Analyzes raw packets and flow metadata from live traffic. Enriched with rule and threat intelligence feeds.	Relies on static signatures and thresholds for detection. Prevention possible with packet dropping.
Snort [44]	Network-based. Rule-driven configuration with inline or passive deployment.	Relies on IP/port signatures and packet headers for pattern matching.	Rules based detection using packet level signatures. Limited rule based prevention with inline blocking.
Wazuh [46]	Host-based. Integrates Elastic Stack for centralized log monitoring and visualization.	Depends on system logs, configuration audits, and vulnerability assessment data.	Detects anomalies through rule/decoder correlation. Has separate active response modules for prevention.
Zeek [48]	Network-based. Event-driven scripting with protocol analysis and session tracking.	Extracts structured metadata from flows. Operates on protocol events and derived features.	Behavioral detection via protocol analysis and feature-based scripting. no native prevention capabilities.
Avantgarde (Proposed)	Host-based with network awareness. Employs multithreaded live packet capture and decision processing.	Combines model predictions and IDS alerts. Independent of static signatures or system logs/calls.	ML Analysis Engine fuses ML and IDS outputs to generate confidence-based intrusion decisions which dynamically adapts firewall rules for prevention.

5.3 Statistical Analysis

5.3.1 Statistical Analysis for Model Evaluation

To evaluate our trained models on their performance and to compare them with existing works we used evaluation metrics like recall (5.3), precision (5.2), f1-score (5.4), and also accuracy (5.1). The dataset we used for the models was BCCC-CIC-IDS2017, which is highly imbalanced. And we trained our models to differentiate between all classes. That is why we chose the Macro metrics to evaluate our models' performance, as it gives a more robust idea of how the models treat each class. But a lot of the existing research only trained their models to detect between Benign and Malicious traffic. This boosts their performance metrics by mitigating the issue of misclassifications between two different attack types. To compare with those models' performance, we have also calculated the metrics from binary classification confusion matrices.

Accuracy: Measures the performance of a model by only taking correctly classified instances into account. This is not a great metric for highly imbalanced datasets like most IDS datasets.

$$\text{Accuracy} = \frac{\sum_{i=1}^C TP_i}{\sum_{i=1}^C (TP_i + FP_i + FN_i)} \quad (5.1)$$

Precision: Measures how many of the classified threats are actually threats. Higher precision means the model has the ability to avoid false alarms.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (5.2)$$

Recall: Measures how many of the actual threat instances were correctly classified. Higher recall means the model has the ability to detect most of the threats correctly.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (5.3)$$

F1-Score: Measures the harmonic mean of precision and recall. A high F1-score means the model has a good balance between avoiding false alarms and correctly detecting threats.

$$\text{F1-score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5.4)$$

Macro_Metrics: Macro-averaged metrics show a model's ability to treat each class equally in avoiding false alarms, correctly classifying threats, and having a good balance between the two.

$$\text{Macro_Precision} = \frac{1}{C} \sum_{i=1}^C \text{Precision}_i \quad (5.5)$$

$$\text{Macro_Recall} = \frac{1}{C} \sum_{i=1}^C \text{Recall}_i \quad (5.6)$$

$$\text{Macro_F1-Score} = \frac{1}{C} \sum_{i=1}^C \text{F1-score}_i \quad (5.7)$$

Here, in the formulae (5.1), (5.5), (5.6), and (5.7), the i stands for current class. While the C in formulae (5.5), (5.6), and (5.7), refers to total number of classes/attack types.

5.3.2 Statistical Model for Decision-Making

The final decision layer of Avantgarde framework uses a statistical model to decide whether or not to Block or Allow an IP based on the independent IDS signatures and MLAE predictions.

Confidence Mapping:

- **Suricata:**
 - **suri_any_mal:** stores binary alerts of whether or not any malicious signals were seen.
 - **suri_ratio:** keeps ratio of a IPs' malicious alerts to total signatures.
- **MLAE:** Stores each model's prediction in **stackingRX**, **votingRX**, and **votingRXC**.
- These features are then linearly combined and scaled between 0 and 1 using a sigmoid function to obtain a probabilistic confidence.

Formula for the Linear_score:

$$\begin{aligned} \text{linear_score} = & \beta_0 + \beta_1 \cdot \text{suri_any_mal} + \beta_2 \cdot \text{suri_ratio} \\ & + \beta_3 \cdot \text{stackingRX} + \beta_4 \cdot \text{votingRX} + \beta_5 \cdot \text{votingRXC} \end{aligned} \quad (5.8)$$

Here in the equation 5.8, each β_i is a coefficient (tunable parameter), and the features represent evidence from Suricata alerts and machine learning predictions.

Formula for confidence:

$$\text{confidence} = \frac{1}{1 + e^{-k \cdot \text{linear_score}}} \quad (5.9)$$

Here in the equation 5.9, k is a steepness parameter that controls the sensitivity of the sigmoid function.

Temporal Smoothing with EMA:

Because the confidence scores can fluctuate significantly between packets, we used the Exponential Moving Average to smooth out the increase/decrease of the confidence score.

$$EMA_t = (1 - \alpha) \cdot EMA_{t-1} + \alpha \cdot confidence_t \quad (5.10)$$

Here in the equation 5.10, $\alpha \in [0, 1]$, controls the smoothing factor, to prevent decision changes due to short-term fluctuations.

5.4 Comparisons and Relationships

We have carried out qualitative and quantitative analysis on the relevant research available to make the understanding of our research objectives clear. The qualitative comparison focuses on the research domains, goals, datasets and techniques used while also depicting if their proposed study was applied or if it remained conceptual. The quantitative analysis focuses completely on performance metrics. Where the ML models from research the works are compared especially based on recall, precision, f1-score, and accuracy which help to determine the effectiveness of variety of models used in different research works. Complete analysis of these research works is essential to understand how the proposed framework can be better by improving on prior work, addressing limitations and achieving better results.

5.4.1 Qualitative Analysis

From the initial stages of the research, we have reviewed, taken inspiration, and tried to improve upon and contribute to existing research in the field. This list compiles the domains of the reviewed research papers, the datasets and methodologies used, and whether or not the proposed ideas were implemented.

Table 5.13: Qualitative Analysis of Reviewed Papers

Study (Year)	Domain	Primary Goal	Techniques / Models / Solutions	Dataset / Environment	Contribution	Applied
Biplob et al. (2024) [27]	Bangladesh's banking sector security	Argument for ML/DL in Banking security	ML/DL analytics	Not specified	Highlighted ML/DL role in securing digital finance.	✗
Ferrag et al. (2024) [38]	GenAI in security	Review LLM uses/risks	LLMs (ChatGPT, LLaMA, Gemini)	Not specified	Survey of LLM applications in cybersecurity, highlighting vulnerabilities	✗
Goldschmidt & Kucera (2024) [29]	DDoS detection	Real-time extraction	Windower + KILNet (RMSE)	Live traffic	Proposed "Windower" real-time DDoS detection via window-based feature extraction	✓
Mankar et al. (2024) [30]	Malicious URL detection	ML for URL filtering	RF, ExtraTrees, DT, KNN, AdaBoost, NB	URL datasets	Demonstrated ensemble tree ML models outperform others in URL detection	✓
Manzoor et al. (2024) [31]	SMEs security	Evaluate open-source SIEMs	Comparative benchmarking	Simulated SME network	Comparative evaluation of multiple open-source SIEMs, on simulated SME networks.	✓
Siddiqui et al. (2024) [34]	Home networks	Unified lightweight UTM	Firewall + IDPS for anomaly detection & prevention	Real deployment context	Proposed lightweight modular UTM optimized for small home network use.	✓
Pissandis & Demertzis (2023) [25]	Open XDR	AI/ML in XDR	AI-enhanced SIEM/EDR/IDS	Not specified	Applied AI/ML to enhance Open XDR for real-time threat correlation	✗
Schmitt (2023) [26]	AI malware/IDS	Evaluate AI	Gradient Boosting, DL, Super-learners	Network, IoT, Android	Showed AI-driven IDS outperforms classical, suggested super-learners	✓
Nicholls et al. (2021) [18]	Financial cyber-crime	Survey DL/ML	GNNs, GAD	Not specified	Survey linking GNNs and group anomaly detection to financial fraud	✗
Podder et al. (2021) [19]	DL for cybersecurity	Survey & compare DL	DNN, RBM, CNN, RNN, AE	Prior works	Comprehensive review of DL models highlighting RBM and LSTM strengths	✗
Elsayed et al. (2020) [16]	SDN anomaly detection	Hybrid IDS	LSTM-AE + OC-SVM	SDN dataset	Hybrid LSTM-AE+OC-SVM for scalable anomaly detection in SDN	✓
Saad et al. (2019) [13]	UTM on Cloud Network	Incident detection	BLSTM, KNN	10k-sample UTM dataset	Applied BLSTM with PCA for improved incident detection in cloud UTM	✓
Vinaykumar et al. (2019) [14]	Hybrid IDS (NIDS + HIDS)	DL based approach for Intelligent NIDS & HIDS	Deep learning-based IDS	KDDCup99, NSL-KDD, CIC-IDS, ADFA-LD	Hybrid IDS combining NIDS + HIDS with GPU scaling for real-time detection.	✓
Arifin et al. (2017) [8]	Small organization networks	Deploy & test UTM	Endian UTM	Virtualized lab, attacks	Demonstrated feasibility of Endian (UTM) in small networks.	✓
Buczak & Guven (2015) [5]	IDS ML/DM survey	Compare IDS	DT, RF, ANN, clustering, ensembles	DARPA, KDD'99	Classical survey comparing misuse vs anomaly IDS with ML/DM methods	✗
Jones et al. (2007) [2]	Prototyping method	OSS prototyping	Patchwork Prototyping	Case studies Alpha / Beta	Introduced Patchwork Prototyping using OSS for rapid system prototyping	✓
Our Proposed Framework (2025)	Malicious Network Traffic Detection & Prevention	ML & Independent IDS integration for enhanced threat detection	RandomForest, XGBoost, CatBoost, Voting, & Stacking	BCCC-CIC-IDS2017	Fusion of Hybrid ensemble ML models with independent IDS for cross-validation	✓

From the analysis in table 5.13, one thing we can be certain of is that all of these studies are correlated in their shared pursuit of strengthening network security, even through different methodology and focus. Papers from **Manzoor et al. (2024)** [31] and **Arifin et al. (2017)** [8] give more importance to the deployment of solutions

which are open-source/low-cost. Whereas, studies from papers like **Vinaykumar et al. (2019)** [14] and **Saad et al. (2019)** [13] focused on strengthening detection accuracy by shifting towards deep learning & neural networks. Many experimental works have been built upon the understanding established through reviews and surveys conducted by research from papers like **Podder et al. (2021)** [19] and **Buczak & Guven (2015)** [5]. There are also research works which focus explicitly on home networks, DDoS detection and overall contribution of generative AI in papers such as **Siddiqui et al. (2024)** [34], **Ferrag et al. (2024)**[38] and **Goldschmidt & Kucera (2024)**[29]. This type of research shows how the overall landscape for intrusion detection has evolved to detect evolving threat types and their environments as well. The framework that we have proposed, recognizes all of these efforts by merging machine learning based IDS with independent open-source IDS for cross validation and tries to scale itself as an applicable solution with advanced learning methods.

5.4.2 Quantitative Analysis

As most network traffic datasets are highly imbalanced, comparing model performance solely on accuracy score doesn't communicate the model's capabilities. Thus we have built a comprehensive comparison table to compare our model's recall, precision, and f1-scores with previous research works to show it's threat detection capabilities.

Table 5.14: Comparison of related works on IDS using ML/DL

Study (Year)	Dataset	Models / Techniques	Metrics	Classification Type
Gutiérrez-Galeano et al. (2025) [39]	BCCC-CIC-IDS2017	T5-small, fine-tuned with the SimpleT5 wrapper	• Accuracy: 99.90% • Precision: 99.90% • Recall: 99.90% • F1-score: 99.90%	Binary Classification
Wang & Wang (2025) [45]	CIC-IDS2017	Genetic Algorithm + Back Propagation Neural Network (GA-BPNN)	• Accuracy: 98% • Precision: 98.80% • Recall: 95% • F1-Score: 96.50%	Binary Classification
Xu & Liu (2025) [47]	CIC-IDS2017	Multi-Layer Perceptron, CNN	• Accuracy: 97.75% • Precision: 98.94% • Recall: 90.36% • F1-Score: 94.46%	Binary Classification
Soumik (2024) [35]	CIC-IDS2017	Decision Tree, Stacked LSTM, AdaBoost, Random Forest	• Accuracy: 99.90% • Precision: 99.87% • Recall: 99.59% • F1-Score: 99.32%	Binary Classification
Ferrag et al. (2020) [15]	CIC-IDS2017, BoT-IoT	Decision Tree & Rules-based Approach	• CIC-IDS2017_Accuracy: 96.66% • CIC-IDS2017_Macro_Recall: 94.47% • BoT-IoT_Accuracy: 96.99% • BoT-IoT_Macro_Recall: 95.17%	Multi-class Classification
Saad et al. (2019) [13]	10k Time series input samples recorded over UTM platform (Private Dataset)	BLSTM, KNN	• Accuracy: 98.60% • Recall: 99.70%	Binary Classification
Our Proposed Models (2025)	BCCC-CIC-IDS2017	• StackingRX • R - Random Forest • X - XGBoost	• Macro_Accuracy: 99.73% • Macro_Precision: 92.44% • Macro_Recall: 96.75% • Macro_F1-Score: 94.22% • Binary_Accuracy: 99.90% • Binary_Precision: 99.93% • Binary_Recall: 99.96% • Binary_F1-Score: 99.94%	Multi-class Classification

Here, in the comparison table 5.14, we can see our model StackingRX out performs the LLM based model of **Gutiérrez-Galeano et al.(2025)** [39] in all metrics except for Accuracy, which is the only other work we could find on the BCCC-CIC-IDS2017 dataset. On other similar datasets, we can see our StackingRX model outperforms or equals **Wang & Wang (2025)**'s [45] GA-BPNN based model, **Xu & Liu (2025)**'s [47] MLP & CNN based solution and **Soumik (2024)**'s [35] DT, and Random Forest based approach. **Ferrag et al. (2020)** [15] used CIC-IDS2017, and showed the accuracy and macro-recall of their model's performance in multi-class classification. Which is surpassed by our StackingRX model with higher accuracy and macro-recall. In their research, **Saad et al.(2019)** [13] used a private dataset,

and their BLSTM, KNN based models' performance was surpassed by our model in every metrics as well.

5.5 Discussions

5.5.1 Interpretations

The network capture module of the framework captures network traffic, filters out noise with Scapy BPF filters, verifies the host identity of the IPs, assign them segmentations which have different access parameters and finally uses pre-processing steps to create features from the traffic to feed into the Machine Learning Analysis Engine.

The integration of the best performing ML models trained and tested on the BCCC-CIC-IDS2017 dataset, the models were integrated into the Avantgarde framework's Machine Learning Analysis Engine (MLAE), where the models predictions and alerts from the signature based independent IDS Suricata are used to create a confidence score which is dubbed as "fusion decision", it is based on linear transformation, sigmoid mapping and temporal smoothing to enhance the threat detection capability of the proposed HIDPS.

The proposed framework's fusion decision method uses a confidence driven approach. Where it maps each model's output and Suricata signals into confidence scores. From where these raw signals are converted into features, and a linear scoring with a sigmoid mapping produces a normalized confidence value between 0 and 1. Where the higher values imply the likelihood of malicious traffic. To ensure stability in the decision making, this raw confidence is then smoothed using Exponential Moving Average (EMA). By maintaining a running smoothed confidence, the system avoids rapid fluctuations and ensures that temporary noise or short-lived anomalies do not cause frequent shifts between Allow and Block states.

Furthermore, the framework incorporates hysteresis and cooldown period as safeguards. Where hysteresis makes sure the system requires a stronger signal to change states back from Block to Allow, this reduces the risk of jitter when confidence values hover near the thresholds. And the cooldown period enforces a minimum time between decision flips to prevent repeated toggling between Allow and Block states.

This combination of ML integration with independent IDS, confidence-based scoring, temporal smoothing, and safeguarding thresholds create an adaptive and reliable HIDPS for real time threat detection and prevention.

5.5.2 Implications

The overall research is divided into aspects which are technical and contextual. On the technical side of things, the prototype ensures overall feasibility by fusion of ML-based anomaly detection with signature-based IDS in a HIDPS. The overall framework sets itself as a robust, adaptable, and straightforward HIDPS by overall ensuring confidence calibration, smoothing and uncertainty quantification. In previous works, a limitation could be seen as they were limited to using binary classification or standalone ML experiments, while the framework we proposed channelize ML to a real-time prevention loop that directly enforces firewall rules & maintains audit logs. In the research context, the overall framework contributes to intrusion detection by integrating ML models, conformal safety layers and zero-trust principles. The prototype points out that high accuracy metrics alone are not enough, but assurance of resilience and reliability is also needed. This projects the framework as a practical deployment of ML model and zero-trust HIDPS in environments in which adaptability is important. The prototype improves upon previous works by providing extension for classification to multi-class threats, implementing ML hybrid fusion with IDS, and also providing temporal resilience.

5.5.3 Limitations

While the proposed framework demonstrates a promising fusion of machine learning-based anomaly detection and signature-driven intrusion detection from an external source, several limitations must be acknowledged.

Controlled testing environment: The evaluation has so far been performed in a controlled and limited test environment, using simulated malicious behaviors and standard traffic traces. This restricted scope does not encompass the full diversity of real-world network conditions, user behaviors, and attack vectors. Consequently, the system’s robustness against more sophisticated, multi-stage, or evasive attack strategies remains to be validated in a wider deployment scenario.

Database performance and reliability: The prototype employs SQLite in Write-Ahead Logging (WAL) mode for concurrent read/write access. While sufficient for prototyping, SQLite poses scalability challenges under heavy traffic loads and remains vulnerable to corruption in edge cases of abrupt termination or resource contention. For production-scale deployments, more resilient database backends may be necessary.

Real-time processing bottlenecks: The framework implements continuous packet capture and parallel processing for real-time decision making. During high-volume traffic scenarios, this may create computational bottlenecks across packet capture and ML inference. Although multi-threading alleviates some of the load, performance trade-offs are expected in heavily demanding circumstances.

Model drifting and generalization: Due to the constant advancement of threats in the cybersecurity landscape, even after the overall training of the machine learning modules on an overarching dataset, the overall learning remains limited, which can make the system open to model drifting when revealed to novel or rare traffic patterns not represented in training. Without ongoing retraining or adaptive learning, detection accuracy may degrade over time, particularly in dynamic or adversarial network conditions.

Chapter 6

Conclusion

6.1 Summary of Findings

The Avantgarde Framework works with a hybrid approach, which uses independent IDS along with machine learning based IDS to detect and prevent intrusion. The prototype system built showcasing the framework is prepared with modular components like network data flow manager, independent IDS tool, MLAE module, and user interface. This means modules can be swapped or extended, such as changing the external IDS tool or updating ML models for future proofing and continued adaptability. The ML models trained for the Avantgarde framework also show improved metrics in controlled testing, compared to previously existing research works. Alongside the unique fusion decision method for integrating external IDS alerts with ML models predictions for adaptive threat detection and prevention builds a reliable host-based intrusion detection & prevention system, which takes inspiration from existing research works and opens up a new horizon for network security research.

6.2 Concluding Remarks

The proposed framework Avantgarde paves the way towards a more secure internet and network security by integrating new and innovative technologies which enhances threat detection and prevention, overall increasing the robustness and resilience of a network. Despite the challenges, its ability to learn, automate and adapt makes it viable for providing security to networks in real time makes it a worthy consideration for any HIDPS implementation. With further refinement and advancement in its constituting technologies and features, Avantgarde has the potential to take the shape of a standard robust and resilient framework for providing security to any host endpoint in any network.

6.3 Recommendations for Future Work

- **Adaptive Model Retraining:** Build mechanisms to periodically retrain or fine tune the MLAE models on new varieties of packet flows to adapt to evolving attack patterns.
- **Dynamic Threshold Learning:** Instead of fixed thresholds, learn and adapt decision boundaries according to traffic context such as payload rate during different times, host behavior.
- **Conformal Prediction Gate:** Adding a conformal safety layer into the decision loop will help the system abstain on blocking or allowing anomalous, out-of-distribution traffic and avoid overconfident wrong decisions.
- **Adversarial Evasions Resistance:** Explore defenses against evasion techniques such as inputs being specifically perturbed to trick ML models prediction outputs.
- **Deception Integration:** Integrate network deception techniques such as deploying decoys, honeypots, or misleading information within the network environment, creating a multi-layered security approach, potentially luring attackers towards fake assets or revealing their presence.

Bibliography

- [1] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324.
- [2] M. Twidale, “Patchwork prototyping: A rapid prototyping technique that harnesses the power of open-source software,” 2007. [Online]. Available: https://www.academia.edu/2830135/Patchwork_Prototyping_A_Rapid_Prototyping_Technique_That_Harnesses_the_Power_of_Open_Source_Software.
- [3] K. Letou, D. Devi, and Y. J. Singh, “Host-based intrusion detection and prevention system (hidps),” *International Journal of Computer Applications*, vol. 69, no. 26, pp. 27–33, May 2013, ISSN: 0975-8887. DOI: 10.5120/12136-8419.
- [4] N. Moustafa and J. Slay, “Unsw-nb15: A comprehensive data set for network intrusion detection systems (unsw-nb15 network data set),” in *2015 Military Communications and Information Systems Conference (MilCIS)*, 2015, pp. 1–6. DOI: 10.1109/MilCIS.2015.7348942.
- [5] A. L. Buczak and E. Guven, “A survey of data mining and machine learning methods for cyber security intrusion detection,” *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016. DOI: 10.1109/COMST.2015.2494502.
- [6] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ACM, 2016, pp. 785–794. DOI: 10.1145/2939672.2939785.
- [7] N. Moustafa and J. Slay, “The evaluation of network anomaly detection systems: Statistical analysis of the unsw-nb15 data set and the comparison with the kdd99 data set,” *Information Security Journal: A Global Perspective*, vol. 25, no. 1-3, pp. 18–31, 2016. DOI: 10.1080/19393555.2015.1125974.
- [8] F. Arifin, G. Mutiara, and I. Ismail, “Implementation of management and network security using endian utm firewall,” *IJAIT (International Journal of Applied Information Technology)*, vol. 1, p. 43, Aug. 2017. DOI: 10.25124/ijait.v1i02.874.
- [9] N. Moustafa, G. Creech, and J. Slay, “Big data analytics for intrusion detection system: Statistical decision-making using finite dirichlet mixture models,” in *Data Analytics and Decision Support for Cybersecurity: Trends, Methodologies and Applications*, I. P. Carrascosa, H. K. Kalutarage, and Y. Huang, Eds. Cham: Springer International Publishing, 2017, pp. 127–156. DOI: 10.1007/978-3-319-59439-2_5. [Online]. Available: https://doi.org/10.1007/978-3-319-59439-2_5.

- [10] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, “Catboost: Unbiased boosting with categorical features,” in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31, Curran Associates, Inc., 2018. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2018/file/14491b756b3a51daac41c24863285549-Paper.pdf.
- [11] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *4th International Conference on Information Systems Security and Privacy (ICISSP)*, Portugal, Jan. 2018. [Online]. Available: <https://www.scitepress.org/papers/2018/66398/66398.pdf>.
- [12] N. Moustafa, J. Slay, and G. Creech, “Novel geometric area analysis technique for anomaly detection using trapezoidal area estimation on large-scale networks,” *IEEE Transactions on Big Data*, vol. 5, no. 4, pp. 481–494, 2019. DOI: 10.1109/TBDATA.2017.2715166.
- [13] M. M. Saad, T. Iqbal, H. Ali, M. F. Bulbul, S. Khan, and C. Tanougast, “Incident detection over unified threat management platform on a cloud network,” in *2019 10th IEEE International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications (IDAACS)*, vol. 2, 2019, pp. 592–596. DOI: 10.1109/IDAACS.2019.8924299.
- [14] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, “Deep learning approach for intelligent intrusion detection system,” *IEEE Access*, vol. 7, pp. 41 525–41 550, 2019. DOI: 10.1109/ACCESS.2019.2895334.
- [15] M. A. Ferrag, L. Maglaras, A. Ahmim, M. Derdour, and H. Janicke, “Rdtids: Rules and decision tree-based intrusion detection system for internet-of-things networks,” *Future Internet*, vol. 12, p. 44, Mar. 2020. DOI: 10.3390/fi12030044.
- [16] M. Said Elsayed, N.-A. Le-Khac, S. Dev, and A. D. Jurcut, “Network anomaly detection using lstm based autoencoder,” in *Proceedings of the 16th ACM Symposium on QoS and Security for Wireless and Mobile Networks*, ser. Q2SWinet ’20, Alicante, Spain: Association for Computing Machinery, 2020, pp. 37–45, ISBN: 9781450381208. DOI: 10.1145/3416013.3426457. [Online]. Available: <https://doi.org/10.1145/3416013.3426457>.
- [17] S. Tamy, H. Belhadaoui, N. Rabbah, and M. Rifi, “Cyber security based machine learning algorithms applied to industry 4.0 application case: Development of network intrusion detection system using hybrid method,” *Journal of Theoretical and Applied Information Technology*, vol. 98, pp. 2078–2091, Jun. 2020.
- [18] J. Nicholls, A. Kuppa, and N.-A. Le-Khac, “Financial cybercrime: A comprehensive survey of deep learning approaches to tackle the evolving financial crime landscape,” *IEEE Access*, vol. 9, pp. 163 965–163 986, 2021. DOI: 10.1109/ACCESS.2021.3134076.
- [19] P. Podder, S. Bharati, M. R. H. Mondal, P. K. Paul, and U. Kose, “Artificial neural network for cybersecurity: A comprehensive review,” *CoRR*, vol. abs/2107.01185, 2021. arXiv: 2107.01185. [Online]. Available: <https://arxiv.org/abs/2107.01185>.

- [20] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, “Netflow datasets for machine learning-based network intrusion detection systems,” in *Big Data Technologies and Applications*, Z. Deze, H. Huang, R. Hou, S. Rho, and N. Chilamkurti, Eds., Springer International Publishing, 2021, pp. 117–135, ISBN: 978-3-030-72802-1. DOI: 10.1007/978-3-030-72802-1_9. [Online]. Available: https://doi.org/10.1007/978-3-030-72802-1_9.
- [21] M. F. H. Tarique M Barkatullah Tawhidur Rahman and M. S. Islam, “Bangladesh cyber threat landscape 2022,” May 2022, pp. 1–53. [Online]. Available: https://ictd.portal.gov.bd/sites/default/files/files/ictd.portal.gov.bd/publications/effc311d_5097_46ba_afa4_5f44b60a93e6/Bangladesh%20Cyber%20Threat%20Landscape%202022.pdf.
- [22] M. Hossain, M. Jahan, R. Rifat, T. Annajiat, A. A. Rasel, and M. Rahman, “Cyber threats and scams in fintech organizations: A brief overview of financial fraud cases, future challenges, and recommended solutions in bangladesh,” Jan. 2023. DOI: 10.1109/ICIMCIS56303.2022.10017467.
- [23] M. J. Hossain, U. N. Jahan, R. H. Rifat, A. A. Rasel, and M. A. Rahman, “Classifying cyberattacks on financial organizations based on publicly available deep web dataset,” in *2023 International Conference On Cyber Management And Engineering (CyMaEn)*, 2023, pp. 108–116. DOI: 10.1109/CyMaEn57228.2023.10050921.
- [24] B. I. Ismail, S. Abdul, S. M. Khan, S. A. Sattar, and S. Muhammad, “Ai for cyber security: Automated incident response systems,” *Journal of Cybersecurity and Artificial Intelligence*, vol. 2, no. 1, pp. 580–599, Apr. 2023, ISSN: 2959-6386.
- [25] D. Pissanidis, *Integrating ai/ml in cybersecurity: An analysis of open xdr technology and its application in intrusion detection and system log management*, Dec. 2023. DOI: 10.20944/preprints202312.0205.v1.
- [26] M. Schmitt, “Securing the digital world: Protecting smart infrastructures and digital industries with artificial intelligence (ai)-enabled malware and intrusion detection,” *Journal of Industrial Information Integration*, vol. 36, p. 100520, Dec. 2023, ISSN: 2452-414X. DOI: 10.1016/j.jii.2023.100520. [Online]. Available: <http://dx.doi.org/10.1016/j.jii.2023.100520>.
- [27] M. B. Biplob, “Empowering bangladesh’s financial infrastructure, machine learning, and deep learning perspectives on banking cybersecurity,” *Preprints*, Sep. 2024. DOI: 10.20944/preprints202409.0242.v1. [Online]. Available: <https://doi.org/10.20944/preprints202409.0242.v1>.
- [28] D. Celeny, L. Maréchal, E. Rousselot, A. Mermoud, and M. Humbert, *Prioritizing investments in cybersecurity: Empirical evidence from an event study on the determinants of cyberattack costs*, 2024. arXiv: 2402.04773 [q-fin.GN]. [Online]. Available: <https://arxiv.org/abs/2402.04773>.
- [29] P. Goldschmidt and J. Kučera, “Windower: Feature extraction for real-time ddos detection using machine learning,” in *NOMS 2024-2024 IEEE Network Operations and Management Symposium*, 2024, pp. 1–10. DOI: 10.1109/NOMS59830.2024.10575699.

- [30] N. P. Mankar, P. E. Sakunde, S. Zurange, A. Date, V. Borate, and Y. K. Mali, "Comparative evaluation of machine learning models for malicious url detection," in *2024 MIT Art, Design and Technology School of Computing International Conference (MITADTSOciCon)*, 2024, pp. 1–7. DOI: 10.1109/MITADTSOciCon60330.2024.10575452.
- [31] J. Manzoor, A. Waleed, A. F. Jamali, and A. Masood, "Cybersecurity on a budget: Evaluating security and performance of open-source siem solutions for smes," *PLOS ONE*, vol. 19, no. 3, pp. 1–24, Mar. 2024. DOI: 10.1371/journal.pone.0301183. [Online]. Available: <https://doi.org/10.1371/journal.pone.0301183>.
- [32] H. Satılmış, S. Akleyek, and Z. Y. Tok, "A systematic literature review on host-based intrusion detection systems," *IEEE Access*, vol. 12, pp. 27 237–27 257, Feb. 2024, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3367004.
- [33] J. P. Seara and C. Serrão, "Automation of system security vulnerabilities detection using open-source software," *Electronics*, vol. 13, no. 5, 2024, ISSN: 2079-9292. DOI: 10.3390/electronics13050873. [Online]. Available: <https://www.mdpi.com/2079-9292/13/5/873>.
- [34] A. Siddiqui, B. Rimal, M. Reisslein, G. Deepak, and Y. Wang, "Sutms: Designing a unified threat management system for home networks," *IEEE Access*, vol. PP, pp. 1–1, Jan. 2024. DOI: 10.1109/ACCESS.2024.3410111.
- [35] M. S. Soumik, "A comparative analysis of network intrusion detection (nid) using artificial intelligence techniques for increase network security," p. 102, Dec. 2024. DOI: <https://doi.org/10.30574/ijrsra.2024.13.2.2664>.
- [36] G. Ali, S. Shah, and M. ElAğendi, "Enhancing cybersecurity incident response: Ai-driven optimization for strengthened advanced persistent threat detection," *Results in Engineering*, vol. 25, p. 104 078, Jan. 2025, ISSN: 2590-1230. DOI: 10.1016/j.rineng.2025.104078.
- [37] M. B. Al-Doori and K. M. A. Alheeti, "Ai-driven features for intrusion detection and prevention using random forest," *Journal of Cybersecurity and Information Management (JCIM)*, vol. 16, no. 01, pp. 1–14, Feb. 2025, ISSN: 2790-8010. DOI: 10.54216/JCIM.160101.
- [38] M. A. Ferrag et al., *Generative ai in cybersecurity: A comprehensive review of llm applications and vulnerabilities*, 2025. arXiv: 2405.12750 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2405.12750>.
- [39] L. Gutiérrez-Galeano, J.-J. Domínguez-Jiménez, J. Schäfer, and I. Medina-Bulo, "Llm-based cyberattack detection using network flow statistics," *Applied Sciences*, vol. 15, no. 12, 2025, ISSN: 2076-3417. DOI: 10.3390/app15126529. [Online]. Available: <https://www.mdpi.com/2076-3417/15/12/6529>.
- [40] Open Information Security Foundation (OISF), *Suricata documentation*, Accessed: 2025-10-04, OISF / Suricata Project, 2025. [Online]. Available: <https://suricata.io/documentation/>.
- [41] OSSEC Project, *Ossec documentation*, Accessed: 2025-10-04, OSSEC Foundation, 2025. [Online]. Available: <https://www.ossec.net/docs/>.

- [42] N. J. Palatty, “90+ cyber crime statistics 2025: Cost, industries & trends,” N. James, Ed., Astra Inc., 2025, p. 3. [Online]. Available: <https://www.getastra.com/blog/security-audit/cyber-crime-statistics/>.
- [43] M. Shafi, A. H. Lashkari, and A. H. Roudsari, “Ntlflowlyzer: Towards generating an intrusion detection dataset and intruders behavior profiling through network and transport layers traffic analysis and pattern extraction,” *Computers & Security*, vol. 148, p. 104 160, 2025, ISSN: 0167-4048. DOI: <https://doi.org/10.1016/j.cose.2024.104160>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404824004656>.
- [44] Snort Project, *Snort documentation and setup guides*, Accessed: 2025-10-04, Snort / Cisco / Talos, 2025. [Online]. Available: <https://www.snort.org/documents>.
- [45] J. Wang and X. Wang, “An optimization model of computer network security based on gabp neural network algorithm,” *EURASIP Journal on Information Security*, vol. 2025, no. 14, 2025. DOI: 10.1186/s13635-025-00196-5. [Online]. Available: <https://doi.org/10.1186/s13635-025-00196-5>.
- [46] Wazuh Project, *Wazuh documentation*, Accessed: 2025-10-04, Wazuh, Inc., 2025. [Online]. Available: <https://documentation.wazuh.com/current/index.html>.
- [47] Z. Xu and Y. Liu, *Robust anomaly detection in network traffic: Evaluating machine learning models on cicids2017*, Jun. 2025. DOI: 10.48550/arXiv.2506.19877.
- [48] Zeek Project, *Zeek documentation (book of zeek)*, Accessed: 2025-10-04, Zeek / Corelight, 2025. [Online]. Available: <https://docs.zeek.org/en/master/>.
- [49] *Online vs offline consumer-brand interactions 2022 — Statista — statista.com*, <https://www.statista.com/statistics/1221311/interactions-companies-consumers-online-offline/>, [Accessed 29-01-2025].
- [50] *Watch out: Biggest data breaches of 2024, so far — foxnews.com*, <https://www.foxnews.com/tech/watch-out-biggest-data-breaches-2024-so-far>, [Accessed 29-01-2025].